

Prompt-Engineering — compiled path

Compiled from `/Users/darkonikolic/Documents/learning/paths/Prompt-Engineering`. Canonical sources stay in place.

Phase — 01-constraint-and-scope

Directory: `01-constraint-and-scope/`

Source: `01-constraint-and-scope/00-orientation.md`

Prompt Engineering — practical path

What this is

Six modules covering the skills that determine whether AI-assisted development produces usable output or wasted work. Every file solves a concrete problem. No theory.

Works with Claude Code and Cursor — tool-specific notes are labeled [Claude] and [Cursor] where syntax differs.

What you will be able to do

- Write constraints that AI actually follows (not suggestions it ignores)
- Construct context that produces correct output on the first turn
- Apply the right prompt pattern for each task type: feature, bug, refactor, architecture, review
- Detect and recover from the four failure modes before they compound
- Configure workspace files that carry constraints across sessions
- Evaluate output against binary criteria before accepting it

Module map

Module	Directory	Problem it solves
01	<code>01-constraint-and-scope/</code>	AI does too much, too little, or the wrong thing
02	<code>02-context-construction/</code>	Output is wrong because context was wrong
03	<code>03-task-patterns/</code>	Same task produces inconsistent results
04	<code>04-failure-patterns/</code>	Output is wrong and you don't know why or what to do
05	<code>05-workspace-configuration/</code>	Rules and constraints reset every session
06	<code>06-output-evaluation/</code>	You can't tell if output is good until it breaks

Reading order: 01 → 02 → 05 → 03 → 04 → 06. - Start with 01 and 02: these are the foundation — constraints and context determine everything else. - Read 05 early: workspace configuration makes 01 and 02 durable across sessions. - Read 03 when you need patterns for a specific task type — use it as reference. - Read 04 when something goes wrong — use it as a diagnostic. - Read 06 last: output evaluation makes no sense until you have criteria to evaluate against.

How to use this path

Each module ends with a lab. The lab applies the module's concepts to a real task. Do the labs — reading without doing produces familiarity, not skill.

Use the non-lab files as reference. When a prompt fails, come back to module 04. When you're configuring a new project, come back to module 05. The files are dense by design.

What this path does not cover

- Claude Code GSD workflow (discuss → plan → execute → verify → ship) → see Claude-Fast-Track path
- Specification-driven development → see Claude-Fast-Track path
- Agent orchestration, parallel agents → see Claude-Fast-Track path modules 05 and 16

Source: 01-constraint-and-scope/01-must-must-not.md

Must / Must Not — Writing Constraints That Hold

Why soft language fails

“Try to keep it simple” is not a constraint. The model reads it as a preference and weighs it against other signals — the existing codebase style, what seems reasonable, what a senior dev might add. It will lose that competition.

Soft language the model will ignore: - “try to”, “prefer”, “ideally”, “where possible”, “keep it simple”, “avoid if you can”

Hard language the model treats as a rule: - “must”, “must not”, “do not”, “never”, “stop if”, “only”

The difference is not style — it is whether the instruction creates a decision boundary. Soft language gives the model room to decide. Hard language removes that room.

The constraint template

Every constrained prompt needs at minimum:

```
must: [what must be true about the output]
must not: [what the output must never include]
stop if: [condition where the model should halt and ask rather than guess]
```

You do not need all three for every prompt. Use `stop if` whenever the model would need to make a design decision with non-obvious tradeoffs. Use both `must` and `must not` for any task touching more than one file.

Scope constraints vs method constraints

These are different and both matter.

Scope constraint — what to build, what to touch: - “must only modify files under `internal/handler/`” - “must not create new files” - “must not change the public API”

Method constraint — how to build it: - “must not add new dependencies” - “must use the existing `logger` package, not `fmt.Println`” - “must not use reflection”

A prompt with only scope constraints can still produce code that violates your architectural rules. A prompt with only method constraints can still cause unbounded scope creep. You need both for any non-trivial task.

Weak vs strong: adding a feature

Weak:

```
Add rate limiting to the API. Keep it simple and don't over-engineer it.
```

Strong:

```
Add rate limiting to POST /api/tasks.

must: use the existing `middleware/` package pattern
must: store rate limit state in the Redis client already injected into the handler
must not: add new packages or dependencies
must not: modify any handler other than tasks_handler.go
must not: add configuration structs — hardcode the limit as a constant with a TODO comment
stop if: the existing Redis client does not have the methods needed for atomic increment+TTL
```

The weak version will produce rate limiting that adds a new library, touches multiple handlers, and adds a config struct “for flexibility”. The strong version cannot — each of those paths is explicitly closed.

Weak vs strong: fixing a bug

Weak:

```
Fix the nil pointer panic in the user service. Be careful not to break anything.
```

Strong:

```
Fix the nil pointer panic in internal/service/user.go at the GetByEmail method.

must: fix only the nil check on the returned db row
must not: change the method signature
must not: modify the calling code in handler/auth.go
must not: add logging or error wrapping beyond what already exists in this file
stop if: the fix requires changing how the db layer returns errors — ask before proceeding
```

“Be careful not to break anything” is the most common useless constraint in prompts. It has no effect. The model was already trying not to break things. The strong version defines exactly what “not breaking anything” means in this context.

Weak vs strong: refactoring

Weak:

```
Refactor the payment processing code to be cleaner and more maintainable.
```

Strong:

```
Refactor internal/payment/processor.go.

must: extract the charge logic into a private method with a clear name
must: preserve all existing method signatures on PaymentProcessor
must not: change any behavior — refactor only, no logic changes
must not: add new types or interfaces
must not: split into multiple files
must not: change error types returned
stop if: you identify a logic bug — report it as a comment, do not fix it
```

“Cleaner and more maintainable” is a value judgment. The model will make it differently than you would. The strong version specifies exactly what refactoring is permitted and puts a hard stop on the most common failure mode of refactoring tasks: the model finds a real bug and fixes it, mixing refactor and behavior change in the same commit.

The “stop if” instruction

Use `stop if` when: - The model would need to make a non-trivial architectural decision to proceed - The task has a dependency you are not sure exists - There are two valid approaches with meaningfully different tradeoffs - Proceeding incorrectly would be harder to undo than asking upfront

```
stop if: the existing auth middleware does not export the claims struct —
ask whether to add an export or use a different approach
```

Without `stop if`, the model will make that decision silently. You will only discover it when reviewing the output. In Claude Code, an unanswered question means three files changed in a direction you did not choose.

Checklist

- [] Every soft word replaced with must/must not or removed
- [] Scope constraint present: which files/modules may be touched

- [] Method constraint present: what approaches are permitted or forbidden
- [] `stop if` covers any decision point where the model would guess
- [] Constraints are specific enough that a different model would produce the same boundaries

Source: `01-constraint-and-scope/02-non-goals-as-constraint.md`

Non-Goals as Constraint

Why non-goals outperform positive descriptions

Telling the model what to build describes a target. Telling it what not to build describes a boundary. The model stops at boundaries. It drifts toward targets.

A goal statement like “add a GET /tasks endpoint” is complete from your perspective — you know what you mean. The model fills the rest with judgment: “a GET endpoint probably needs filtering, and filtering means query params, and query params means validation, and validation means error messages, and error messages should probably follow the existing pattern in the 400 handler...” None of that was asked for. None of it was prevented.

Non-goals prevent scope creep by forcing an explicit decision. You are not relying on the model to infer the minimum viable version. You are telling it the minimum viable version is the only acceptable version.

The rule: 3 non-goals minimum after every goal

After every goal statement, write at least 3 non-goals — the 3 things a reasonable developer would add while implementing the feature.

Pattern:

```
Goal: [what to build]

Non-goals:
- [reasonable thing someone would add — do not build]
- [reasonable thing someone would add — do not build]
- [reasonable thing someone would add — do not build]
```

Three is the floor. If you can think of four or five, write them. The cost of an unnecessary non-goal is one line of text. The cost of a missing non-goal is an LLM that rewrote your pagination logic “while it was there”.

How to identify non-goals

Ask: “What would a reasonable developer add while implementing this?”

The answer to that question is your non-goal list. Not things that are absurd or irrelevant — things that are reasonable, helpful, and completely out of scope.

A developer implementing GET /tasks would reasonably add: - Filtering by status because tasks lists always need it - Pagination because nobody returns unbounded lists - Sorting because users want it - Auth middleware because this is probably a protected resource - A search param because it's a list endpoint

Every one of those is a non-goal for “add a basic GET /tasks endpoint that returns all tasks”.

This exercise also clarifies your own thinking. If you write “no pagination” and then feel uncomfortable about it, that discomfort means you actually do need to decide on pagination — not leave it for the model to infer.

The scope creep failure mode

The canonical failure: you ask for a feature. The model adds three others. The output is technically correct and probably useful. You accept it because it looks fine. Three days later you're debugging an interaction between the pagination it added and the cursor-based pagination your frontend already had.

The model is not being reckless. It is being helpful by the only definition it has: complete, reasonable code. “Reasonable” means “what a competent developer would do.” A competent developer would add pagination. You needed to say no.

Common scope creep patterns by task type:

Task	What the model adds without being asked
New endpoint	Auth, validation, logging, error handling beyond spec
Bug fix	Refactor of surrounding code, additional defensive checks
Add a field	Migration logic, index, cascading changes to related models
Refactor	Logic improvements, “while I’m here” bug fixes
Add a test	Tests for adjacent untested code, test helpers, fixtures

Example: GET /tasks endpoint

Goal: Add a GET /tasks endpoint that returns all tasks from the database.

Weak prompt (no non-goals):

```
Add a GET /tasks endpoint to the tasks API.
```

Strong prompt (with non-goals):

```
Add a GET /tasks endpoint to internal/handler/tasks_handler.go.

Goal: return all tasks from the database as a JSON array

Non-goals — do not implement:
- filtering by any field
- pagination or cursor logic
- sorting or ordering parameters
- authentication or authorization checks
- search functionality
- response envelope wrapping (return the array directly)

must: match the existing handler pattern in this file
must not: modify any other handler or file
```

The non-goals do not just prevent bad output — they document intent. When someone reviews this code, the non-goals tell them the simplicity was deliberate.

Example: Add email notifications

Goal: Send an email when a task is marked complete.

Non-goals: - Do not add email templates — use a hardcoded string - Do not add an unsubscribe mechanism - Do not add retry logic for failed sends - Do not queue emails — send synchronously in the same request - Do not add email tracking or open pixels

Without these, the model will produce a full email system with templates, a queue, retry with backoff, and a config struct for SMTP settings. All reasonable. None of it asked for.

Claude and Cursor: where to place non-goals

Both tools read non-goals the same way — as explicit constraints. Placement matters.

In Claude Code (single prompt or CLAUDE.md): Put non-goals immediately after the goal statement, before any implementation instructions. The model reads top to bottom and front-loads constraints from earlier in the prompt.

```
Goal: [what to build]

Non-goals:
- ...

Implementation:
[instructions]
```

In Cursor (composer or inline): Same structure. Cursor's composer treats the full prompt as context. Put non-goals before the file-specific instructions so they apply to all files the model might touch.

If you are using a `.cursorrules` file for persistent constraints, add your project-level non-goals there: "this project does not use ORMs", "this project does not add logging inside business logic", "pagination is handled at the API gateway, not in handlers."

Checklist

- [] Every goal statement followed by at least 3 non-goals
 - [] Non-goals are things a reasonable developer would add (not absurd exclusions)
 - [] Non-goals written as explicit "do not" statements, not as soft preferences
 - [] Non-goals cover: related features, defensive additions, "while I'm here" improvements
 - [] Non-goals placed before implementation instructions in the prompt
 - [] Discomfort with a non-goal treated as a signal to make a real decision, not skip it
-

Source: `01-constraint-and-scope/03-scope-signals.md`

Scope Signals — Reading Output Before You Accept It

What scope signals are

A scope signal is a pattern in the output that tells you the prompt was under-constrained. You can detect them before accepting the output, before running the code, and in Claude Code — before the model finishes generating.

Recognizing scope signals is cheaper than fixing the output. Fixing under-constrained output means: reviewing code you didn't ask for, reverting changes that touched the wrong files, and debugging interactions between your code and the model's additions.

Reading output for scope signals is a skill. These are the four patterns to train on.

Signal 1: output touches files not mentioned in the prompt

The model modified, created, or imported from a file you did not reference.

What this looks like: - A new file appears in the diff - An existing file outside your stated scope has changes - The model added an import from a package you didn't mention - A test file was created or modified when you didn't ask for tests

What it means: the model interpreted your goal as requiring more than you specified. It made a judgment call about scope. That judgment may be wrong.

What it does not mean: the changes are bad. Sometimes the model is right that another file needs to change. But "the model decided to touch it" is not the same as "you decided to touch it."

Action: stop. Identify which files were out of scope. Decide intentionally whether those changes are acceptable. If not, restart with explicit file scope constraints.

Signal 2: output adds a feature that "makes sense" but wasn't asked

The model implemented something you didn't request because it logically follows from what you did request.

What this looks like: - You asked for a POST endpoint, the model also added input sanitization - You asked for a bug fix, the model also refactored the function it was in - You asked for a new field, the model also added a getter method - You asked for validation, the model also added error codes you haven't defined elsewhere

What it means: the model is completing what it thinks the feature should look like. It is reasoning about what belongs with the task, not what you specified.

Danger level: high. These additions are often plausible and blend in. You review the output, it looks reasonable, you accept it, and you've now committed to an error handling pattern or a sanitization behavior that you didn't evaluate.

Action: stop before the feature is accepted. Evaluate whether the addition is actually wanted. If yes, make it explicit in your prompt so it is intentional. If no, restart with non-goals that exclude it.

Signal 3: output creates abstractions for hypothetical future use

The model introduced an interface, a config struct, a factory function, or a base class that nothing currently uses but “would be useful when you add more implementations.”

What this looks like: - An interface with one implementor and no other consumers - A `Config` struct with fields for options that don't exist yet - A `New*` constructor that wraps a type you were using directly - Comments like “// extend this to support...” or “// future: add X here”

What it means: the model is optimizing for a codebase it imagines rather than the codebase you have. It is doing speculative architecture.

This is expensive. Speculative abstractions create coupling, increase the surface area of the change, and make the diff harder to review. They also constrain your future decisions — you now have an interface you need to maintain or remove.

Action: reject the abstraction. Restart with “must not add new interfaces/types/abstractions not required by the immediate change.”

Signal 4: output changes behavior for callers not mentioned

The model modified a function, method, or type that has callers you did not reference, in a way that changes behavior for those callers.

What this looks like: - A function signature changed (new parameter, different return type) - Error types changed on a method with multiple call sites - A struct field was renamed or removed - Default behavior changed on something with existing dependents

What it means: the model treated a local change as a refactor opportunity. It may not have checked (or been able to check) all callers.

This is the highest-risk signal. Behavior changes to shared code can break callers in ways that don't surface until runtime. In a compiled language, the compiler catches signature changes. In dynamic languages or across service boundaries, they are invisible until something breaks.

Action: stop immediately. Do not accept. Restart with explicit “must not change the signature or behavior of any existing function” and “must not modify [shared file].”

What to do when you see a signal mid-output

In Claude Code: you can interrupt generation. If you see a scope signal before the model finishes — a new file being created, a function signature changing — stop the generation. Do not let it finish. A complete but wrong output is harder to reason about than an incomplete one.

How to interrupt and restart: 1. Stop generation 2. Identify which signal triggered 3. Add the specific constraint that would have prevented it 4. Restart the prompt with that constraint added

Do not try to salvage a partial output with a follow-up message. “Undo the changes to X” is less reliable than a clean restart with better constraints.

In Cursor: if you see scope signals in the diff preview before applying, reject the change. Cursor shows diffs before application — use that review step. Do not apply and then revert; reject before it touches your files.

Scope audit checklist

Apply this to any output before accepting it:

- ☐ Does the diff include only files I mentioned or expected to change?
- ☐ Does the output implement only what I asked, or did it add features I didn't request?
- ☐ Are there new types, interfaces, or abstractions I didn't ask for?
- ☐ Did any existing function or method signature change?

Four questions. If any answer is “yes, and I didn't intend that” — do not accept. Restart with a tighter prompt.

The checklist is a gate, not a formality. Run it on every non-trivial output. The cost is 30 seconds. The cost of skipping it and accepting out-of-scope changes compounds across every subsequent session that builds on that output.

Checklist

- ☐ Can identify all 4 scope signals by pattern

- [] Know the difference between “model was right to add this” and “I chose to add this”
- [] Have the habit of checking file scope before reading code quality
- [] Know to stop mid-generation rather than accept and revert
- [] Run the 4-question scope audit before accepting any non-trivial output

Source: 01-constraint-and-scope/04-lab-constrain-a-real-prompt.md

Lab — Constrain a Real Prompt

The original prompt

Add input validation to the task creation endpoint

This prompt will produce something. It will probably work. It will also do things you didn't ask for and leave out decisions you needed to make explicitly.

Step 1 — Identify what’s missing

Go through the original prompt and mark every missing constraint:

Missing	Why it matters
No file path	Model picks where to add validation — may add it in the wrong layer
No definition of “valid”	Model decides what rules apply — required fields? length? format?
No constraint on what “valid” means for each field	Model invents rules
No method constraint	Model may use a validation library, add a new struct, or inline checks
No scope boundary	Model may touch the handler, the service, the model, the tests
No non-goals	Model will add: detailed error messages, a validation struct, error codes
No stop condition	Model will guess if it finds ambiguity in the existing code

The original prompt has zero constraints. It is a description of intent, not an instruction. The model will produce its best interpretation of “validation done well” — which includes every decision listed above, made silently.

Step 2 — Rewrite using must / must not / stop-if

Add input validation to the POST /tasks endpoint.

Target file: internal/handler/tasks_handler.go – CreateTask function only

must: validate that the request body contains `title` (non-empty string) and `due\_date` (valid RFC3339 date)

must: return 400 with the existing error response format used in this file if validation fails

must: add validation inline in the CreateTask function – do not create a separate validator type

must not: add a validation library or new dependency

must not: modify the Task model or any file other than tasks_handler.go

must not: add field-level error messages – one error message per request is enough

must not: add tests – validation only

Non-goals:

- do not validate optional fields (assignee_id, tags, description)
- do not add max-length checks unless the existing db schema enforces a constraint
- do not add a reusable validation helper – this is handler-specific logic

stop if: the existing error response format is not consistent in this file – show me what you find and ask which pattern to follow

stop if: due_date validation requires a library not already imported – ask before adding an import

Step 3 — Annotations explaining each choice

```
Add input validation to the POST /tasks endpoint.
# ^ specific endpoint, not "the endpoint"

Target file: internal/handler/tasks_handler.go – CreateTask function only
# ^ scope constraint: one file, one function

must: validate that the request body contains `title` (non-empty string) and `due_date` (valid RFC3339 date)
# ^ definition of "valid" – model cannot invent rules

must: return 400 with the existing error response format used in this file if validation fails
# ^ method constraint: use what exists, don't invent

must: add validation inline in the CreateTask function – do not create a separate validator type
# ^ method constraint: prevents speculative abstraction (Signal 3)

must not: add a validation library or new dependency
# ^ method constraint: hard dependency boundary

must not: modify the Task model or any file other than tasks_handler.go
# ^ scope constraint: one-file boundary (prevents Signal 1)

must not: add field-level error messages – one error message per request is enough
# ^ prevents reasonable-but-unasked feature (prevents Signal 2)

must not: add tests – validation only
# ^ explicit non-goal so tests don't appear in the diff

Non-goals:
- do not validate optional fields (assignee_id, tags, description)
# ^ prevents "while I'm here" validation expansion
- do not add max-length checks unless the existing db schema enforces a constraint
# ^ forces a real decision, not model judgment
- do not add a reusable validation helper – this is handler-specific logic
# ^ prevents abstraction for hypothetical future use

stop if: the existing error response format is not consistent in this file – show me what you find
and ask which pattern to follow
# ^ the model finding inconsistency is a decision point, not a judgment call for it to make

stop if: due_date validation requires a library not already imported – ask before adding an import
# ^ closes the gap between "no new dependency" and "but what if stdlib isn't enough"
```

Step 4 — The experiment

Send both versions. Use the same codebase, same context, same model.

After receiving output from each version, compare:

Dimension	Vague prompt output	Constrained prompt output
Files touched	Count them	Count them
New types or structs	Present?	Present?
New imports	How many?	How many?
Validation rules	What did the model decide?	What you specified
Error format	Invented or existing?	Existing (per constraint)
Tests added	Yes/no	No (per constraint)
New helpers	Yes/no	No (per constraint)

The vague prompt will score higher on almost every “added something” dimension. That is the failure mode, not the success mode.

Step 5 — Apply the scope audit checklist to the constrained output

Before accepting the output from the constrained prompt:

- ☐ Does the diff include only `internal/handler/tasks_handler.go`?
- ☐ Does the output add only inline validation for `title` and `due_date`?
- ☐ Are there any new types, interfaces, or validation structs in the diff?
- ☐ Did any existing function signature change (e.g., `CreateTask`'s parameters or return type)?

If all four pass: the prompt was tight enough. Accept the output.

If any fail: identify which scope signal triggered, add the missing constraint, restart.

Reusable format

Save this as a template for any task with validation or input handling:

```
Add [specific feature] to [specific endpoint or function].

Target file: [path] — [function name] only

must: [rule about what valid means — specific, not "validate inputs"]
must: [rule about error format — use existing pattern, specify which]
must: [method constraint — inline vs helper vs library]
must not: [dependency constraint]
must not: [file scope constraint — what cannot be touched]
must not: [feature exclusion — what the reasonable dev would add]

Non-goals:
- do not [thing 1 that seems related but is out of scope]
- do not [thing 2 that seems related but is out of scope]
- do not [thing 3 that seems related but is out of scope]

stop if: [ambiguity that requires a decision, not a guess]
stop if: [missing dependency or inconsistency the model might discover]
```

Checklist

- ☐ Can identify at least 5 missing constraints in the original prompt
 - ☐ Rewrote the prompt with at least 3 `must`, 3 `must not`, and 2 `stop-if` clauses
 - ☐ Added 3 non-goals that prevent common scope creep for this task type
 - ☐ Sent both versions and compared the diffs
 - ☐ Applied the 4-question scope audit to the constrained output
 - ☐ Saved a reusable template for validation tasks in this codebase
-

Phase — 02-context-construction

Directory: 02-context-construction/

Source: 02-context-construction/01-what-to-include.md

What to Include in Context

File References Beat Inline Text

Paste nothing. Reference paths.

When you paste code into the prompt you create a second copy the model treats as potentially stale. Reference the source of truth instead.

```
# Bad — you've now got two versions in context
Here is my handler:
func CreateTask(w http.ResponseWriter, r *http.Request) {
    // 40 lines
}
Now add validation.

# Good — one source of truth
Add input validation to the CreateTask handler in internal/api/tasks.go
```

[Claude] The agent reads files via the Read tool at session start or inline in a message. Say “read internal/api/tasks.go” or reference it — Claude will read it. Pasting it creates drift the moment the file is edited.

[Cursor] Use @tasks.go or @internal/api/tasks.go in the composer. Cursor fetches the current file state. Pasting forces Cursor to work from a potentially outdated snapshot.

Minimum Required Context for Any Task

Three things. If any one is missing the model fills it in by guessing.

- 1. The relevant file(s)** The actual file(s) the task touches. Not the whole module — the file(s).
- 2. The constraint or spec** What rules apply to this task. Language, library restrictions, error handling style, conventions from the codebase. If it exists in a config or CLAUDE.md, say so; don't re-paste it.
- 3. What “done” looks like** An acceptance criterion or output shape. Not a general description — a specific outcome.

```
# Complete minimum context example
File: internal/api/tasks.go
Constraint: stdlib only, no frameworks, errors wrapped with fmt.Errorf("%w")
Done: PATCH /tasks/:id/complete returns 200 with updated task JSON, 404 if id missing, 400 if already complete
```

That's it. Everything else is noise unless the task touches it.

What to Exclude

Test files — unless you're writing or fixing tests. Including tasks_test.go when adding an endpoint causes the model to consider the test structure when generating code, often producing code that mirrors existing test patterns rather than the actual requirement.

Unrelated modules — if the task is in internal/api, don't include internal/store unless the task explicitly calls into it.

Config files not touched by the task — go.mod, .env, Makefile. These get read and occasionally echo back into generated code (e.g. the model copies a library version number into a comment).

Other endpoints or handlers in the same file — if the file has 10 handlers and you're touching one, say which one. The model will still have the file; you don't need to call attention to the other 9.

The Context Pollution Failure

Passing too many files causes the model to use them even when not needed.

```
# This prompt will produce output that mixes store logic into the handler:
Add PATCH /tasks/:id/complete
@tasks.go @store.go @config.go @middleware.go @auth.go @models.go
```

The model sees `store.go` and treats it as relevant. It will likely add store calls, import patterns, or error handling from `store.go` even if the spec says the handler should be simple.

Pattern: every file you include gets weighted as “probably relevant.” The model doesn't have a way to confirm a file is decorative context.

The fix: include only what the task directly mutates or calls. If `store.go` is needed because the handler calls the store, include it. If not, exclude it.

Minimum Viable Context

The smallest set that lets the model complete the task without guessing.

Test it this way: read back your context prompt and ask — “what would the model have to invent to complete this?” If the answer is “nothing”, you have minimum viable context. If the answer is “the error response format” or “whether to use stdlib or a library”, add that.

```
# Minimum viable — nothing left to guess
Task: add GET /tasks endpoint
File: internal/api/tasks.go (read it)
Constraint: stdlib only, respond with JSON array, empty array not null on no results
Done: returns 200 with []Task JSON, errors return {"error": "..."} with appropriate status
```

```
# Under-specified — model guesses the response format
Task: add GET /tasks endpoint
```

```
# Over-specified — includes irrelevant material
Task: add GET /tasks endpoint
Files: tasks.go, tasks_test.go, store.go, models.go, config.go, main.go, go.mod
```

Tool-Specific Notes

[Claude] At session start, list files you want Claude to read before any task. Claude will read them once and hold them in context. Avoid re-reading mid-session unless the file changes.

```
Read these files before we start:
- internal/api/tasks.go
- internal/store/memory.go
```

[Cursor] Open the relevant files in the editor before starting a composer session. Use `@filename` to pin them. Cursor's context window is smaller — be stricter about exclusions.

Checklist

- [] All file references point to paths, not pasted content
- [] Three elements present: relevant file(s), constraint, done condition
- [] Test files excluded (unless writing tests)

- [] Unrelated modules excluded
- [] Config files excluded unless the task modifies them
- [] Each included file has a reason to be there
- [] Nothing left for the model to guess
- [] [Cursor] Files opened in editor before composer session
- [] [Claude] Files listed for reading at session start

Source: 02-context-construction/02-context-hierarchy.md

Context Hierarchy

Three Levels in Both Tools

Every interaction with Claude Code or Cursor operates across three layers simultaneously. Most context problems come from putting information at the wrong level.

Level 1 — Workspace config	CLAUDE.md / .cursorrules
Level 2 — Session context	spec file, PLAN.md, or explicit session message
Level 3 — Task prompt	the individual message or composer turn

The levels stack. Workspace is background, always present. Session sits on top, active for this work. Prompt is foreground, governs this turn.

What Belongs at Each Level

Level 1 — Workspace Config

Always-true constraints that apply to every task in this project.

- Language and library rules (“stdlib only”, “no ORM”)
- Code style that isn’t enforced by a linter
- Architectural decisions that can’t change (“in-memory store, no DB”)
- Patterns you want repeated everywhere (“errors wrapped with fmt.Errorf”)
- What files/dirs are off-limits

[Claude] Lives in `CLAUDE.md` at the project root. Read automatically at session start.

[Cursor] Lives in `.cursorrules` at the project root. Applied to all composer sessions.

```
# CLAUDE.md / .cursorrules example

## Constraints
- Go stdlib only. No third-party packages.
- All errors: fmt.Errorf("operation: %w", err)
- JSON responses always include a "status" field
- Do not modify internal/store — that layer is frozen
```

Level 2 — Session Context

The spec or plan for the current work unit. Active while you’re implementing a feature or working through a task.

- Endpoint spec (request shape, response shape, status codes)
- Acceptance criteria for this milestone
- Decisions made in earlier turns that must carry forward
- Current file state that matters for the rest of the session

[Claude] Introduce this at the start of a session as a message, or reference a spec file: “Read SPEC.md before we start.” Keep it in a file so it persists.

[Cursor] Add the spec file with `@SPEC.md` at the start of the composer. It stays in context for that session.

```
# SPEC.md (session-level context)

## Task API — Sprint 3 endpoints

PATCH /tasks/:id/complete
- Body: none
- Returns: 200 + updated Task, 404 if not found, 409 if already complete
- Task.CompletedAt set to time.Now()

POST /tasks
- Body: {"title": string, "due": RFC3339}
- Returns: 201 + created Task, 400 on missing title
```

Level 3 — Task Prompt

The individual turn instruction. What you want done right now.

- One specific action
- Any clarification that is specific to this turn only
- A question or a correction

```
# Good task prompt — specific, one action
Implement PATCH /tasks/:id/complete using the spec in SPEC.md.
Handler goes in internal/api/tasks.go.
```

Why Mixing Levels Breaks Things

Putting task-level instructions in .cursorrules / CLAUDE.md

Task-level instructions in workspace config don't disappear when the task is done — they persist and contaminate future tasks.

```
# .cursorrules — wrong
- When implementing endpoints, always check if tasks are already complete before updating.
```

This was true for one endpoint. Now it applies to every future endpoint, whether or not it makes sense. Three weeks later, you're debugging why a new endpoint is checking task completion status.

Rule: if the instruction stops being true when the task is done, it doesn't belong in workspace config.

Putting workspace constraints in the prompt

Constraints typed only in a prompt are lost when context clears or degrades.

```
# Prompt — wrong place for this
Use stdlib only. No third-party packages. Add the PATCH endpoint.
```

Next session, next clear, the stdlib constraint is gone. The model uses a library. You revert it. This happens three times.

Rule: if the constraint is true for every task in this project, it goes in CLAUDE.md or .cursorrules — not the prompt.

How the Levels Interact

Workspace is the background layer. It is always present but low-salience. The model treats it as defaults.

Session context is mid-ground. It narrows what workspace config means for this task. If workspace says “JSON responses always include a status field” and session says “PATCH /tasks/:id/complete returns 200 + Task”, the model combines them.

Prompt is foreground. It gets the most weight. If the prompt explicitly says something that conflicts with session context, the prompt wins.

```
Workspace:  "errors wrapped with fmt.Errorf"
Session:    "PATCH /tasks/:id/complete spec"
Prompt:     "Implement the PATCH endpoint"
```

Result: model implements the endpoint, follows the spec, wraps errors as defined in workspace config. Each level contributes without you restating it.

Concrete Placement Examples

Instruction	Level	Reason
"stdlib only, no frameworks"	Workspace	True forever in this project
"in-memory store, do not add DB"	Workspace	Architectural constraint
"errors: fmt.Errorf with %w"	Workspace	Style rule, always applies
"PATCH /tasks/:id/complete spec"	Session	True for this task, not all tasks
"do not modify internal/store"	Workspace	Frozen layer, always true
"implement the PATCH endpoint"	Prompt	One action, this turn
"return 409 if already complete"	Session	Part of spec, not a global rule
"use the handler I showed you earlier"	Prompt	Turn-specific reference

Checklist

- [] Workspace config contains only always-true constraints
- [] Task-specific instructions are NOT in .cursorrules / CLAUDE.md
- [] Session-level spec is in a file (not just typed in the first message)
- [] [Claude] CLAUDE.md exists and is accurate for this project
- [] [Cursor] .cursorrules exists and doesn't contain stale task instructions
- [] Prompts contain only this-turn instructions
- [] If a constraint was only true for one task, it has been removed from workspace config
- [] The three levels are not redundantly repeating the same constraint

Source: 02-context-construction/03-multi-turn-discipline.md

Multi-Turn Discipline

How Context Degrades

By turn 5, the model is working from a compressed representation of what you said in turn 1. This is not a failure — it is how attention and context windows work. Planning around it is the skill.

What degrades is not the model's capability. What degrades is the fidelity of older messages. Early constraints become lower-salience. Early file state gets replaced by what the model has written. Specifications from turn 1 compete with decisions from turns 3 and 4.

The practical consequence: the model that correctly followed your stdlib constraint in turn 2 may import a package in turn 7 not because it forgot the rule but because the rule is now less salient than everything else in context.

Three Degradation Patterns

1. Constraint Forgetting

A rule that was followed early in the session is violated later.

```
Turn 2: correctly uses stdlib net/http
Turn 6: adds github.com/gorilla/mux

Turn 1: errors wrapped with fmt.Errorf
Turn 5: returns raw errors without wrapping
```

Cause: the constraint was stated once in turn 1. By turn 5-6, the model is attending more to recent exchanges than to the original setup.

Signal: review generated code for the specific constraints in your workspace config. Don't assume compliance persists.

2. Scope Drift

The model gradually expands what it's doing beyond the stated task.

```
Turn 1: "add PATCH /tasks/:id/complete"
Turn 3: adds a CompletedAt field to the store interface
Turn 4: refactors the store to support the new field
Turn 5: updates all existing handlers to use the new store interface
```

None of these individual steps is unreasonable — but the model has gone from “add one endpoint” to “refactor the store layer.”

Signal: output touches files or layers you didn't specify. The model starts making decisions that weren't part of the task.

3. Reference Drift

The model uses an older version of a file — either the one it read at session start or one it generated earlier — rather than the current state.

```
You edited tasks.go after turn 2.
Turn 4 output: the model regenerated a function that was already updated in your edit.
```

Signal: generated code doesn't reflect recent manual edits or re-introduces code you removed.

[Claude] Re-read files explicitly after you make manual edits: “Read internal/api/tasks.go again — I made changes outside this session.”

[Cursor] Re-add `@tasks.go` in the next composer message after manual edits.

The Reset Signal — When to Clear vs Continue

Continue the session when: - The task is progressing and constraints are being followed - You're building incrementally on previous output - No degradation patterns have appeared - Turn count is low (1-4)

Clear and restart when: - The model has violated a constraint (continuing compounds it — the violation is now in context as apparent precedent) - The task has shifted significantly from the original spec - You're past turn 6-8 on a complex task - The model's last response required significant correction

[Claude] `/clear` clears the session. Re-establish context from your files — CLAUDE.md, SPEC.md, relevant source files.

[Cursor] Start a new composer. Re-add `@spec.md` and `@relevant-files`.

The rule: if the session has gone wrong, do not try to fix it with more turns. Clear and restart with clean context.

The Checkpoint Technique

A checkpoint is a single message you write at a stable milestone before context degrades. It must allow a fresh session to resume without reading the conversation history.

When to write one: after each meaningful unit of work (one endpoint implemented and verified, one module completed). Write it before starting the next unit.

What it must contain:


```
## Checkpoint — [date/time or milestone label]

### Verified state
- PATCH /tasks/:id/complete implemented in internal/api/tasks.go
- Returns 200 + Task, 404 if not found, 409 if already complete
- Manual test: curl -X PATCH localhost:8080/tasks/1/complete → 200 ✓
- All existing tests pass

### Active constraints (from CLAUDE.md)
- stdlib only
- errors: fmt.Errorf("operation: %w", err)
- in-memory store, no DB layer

### Files modified this session
- internal/api/tasks.go (added handler)
- internal/api/router.go (registered route)

### Next action
Implement POST /tasks — spec in SPEC.md under "POST /tasks"
```

This message can be pasted verbatim to start a new session. It is the minimum viable context for continuation.

Turn-by-Turn Discipline

Each turn should be as self-contained as possible. The goal is that losing the previous turn does not break the current one.

Good turn structure: - Reference files by path, not by “the file we discussed” - State constraints that matter for this turn, even if stated before - Include the specific acceptance criterion for this turn’s output

```
# Good — self-contained turn
Implement PATCH /tasks/:id/complete in internal/api/tasks.go.
Constraint: stdlib only, errors wrapped with fmt.Errorf.
Done: 200 + updated Task, 404 if not found, 409 if already complete.
```

```
# Fragile — depends on previous turns being remembered
Now add the complete endpoint like we discussed.
```

The second form works at turn 2. At turn 6, the model’s reconstruction of “what we discussed” may differ from what you intended.

When NOT to Continue a Session

The model has already violated a constraint.

If the model used a third-party library when stdlib was required, do not continue in the same session. The violation is now part of the conversation context. Continuing treats the violation as accepted prior art. The model may continue the pattern.

Clear. Revert the change. Start fresh with clean context.

The model’s last output required major rewriting.

If you had to rewrite 30+ lines of the model’s output, the session has drifted. The model’s internal representation of what “good output” looks like for this task has diverged from yours. More turns will produce more output that needs rewriting.

Clear. Restate what was done, what was wrong, and what the actual requirement is.

Checklist

- ☐ Degradation patterns identified: constraint forgetting, scope drift, reference drift
- ☐ Files re-read after manual edits before continuing
- ☐ Session cleared when a constraint violation occurred
- ☐ Checkpoint written at each stable milestone
- ☐ Checkpoint contains: verified state, active constraints, modified files, next action

- [] Individual turns are self-contained — no “like we discussed” references
- [] Sessions past turn 6-8 on complex tasks are checkpointed and optionally restarted
- [] [Claude] /clear used when session has gone wrong, not just when context is long
- [] [Cursor] New composer started rather than continuing after a failed direction

Source: 02-context-construction/04-lab-build-context-for-task.md

Lab: Build Context for a Task

The Task

```
Add PATCH /tasks/:id/complete endpoint to task-api
Language: Go, stdlib only, in-memory store
```

This is the raw task. It is under-specified. Your job is to build the context that makes it executable.

Step 1 — Identify What Context Is Required

Work through the three required elements.

Relevant files — what does this task touch?

The handler file. The router where routes are registered. The store if the handler calls it. The Task model if CompletedAt needs to be added.

```
internal/api/tasks.go      - handler goes here
internal/api/router.go     - route registration
internal/store/memory.go   - in-memory store (read to understand Update interface)
internal/models/task.go    - Task struct (does it have CompletedAt?)
```

Do not include: `main.go`, `go.mod`, test files, config files.

Constraint — what rules apply?

From workspace config (CLAUDE.md): - stdlib only, no third-party packages - errors: `fmt.Errorf("operation: %w", err)` - JSON responses include a `"status"` field

Specific to this task: - do not modify the store interface signature — add methods if needed, don't change existing ones - in-memory store only, no persistence

Done condition — what does success look like?

```
PATCH /tasks/:id/complete
- 200 + updated Task JSON if successful
- 404 + {"status": "error", "error": "task not found"} if id not in store
- 409 + {"status": "error", "error": "already complete"} if Task.CompletedAt != zero
- Task.CompletedAt set to time.Now() on success
```

Step 2 — Identify What to Exclude and Why

File	Include?	Reason
internal/api/tasks.go	Yes	Handler lives here
internal/api/router.go	Yes	Route must be registered
internal/store/memory.go	Yes	Handler calls into store
internal/models/task.go	Yes	Need to verify Task struct has CompletedAt
internal/api/tasks_test.go	No	Not writing tests this turn
main.go	No	Not touched by this task
go.mod	No	Not touched; no new dependencies
internal/middleware/	No	Not called by this endpoint
.env / config.go	No	No config changes

Rule applied: every included file has a direct reason. Every excluded file has no direct connection to the task.

Step 3 — Write the Context Message

[Claude] Context Message

```
Read these files before we start:
- internal/api/tasks.go
- internal/api/router.go
- internal/store/memory.go
- internal/models/task.go

Task: Add PATCH /tasks/:id/complete handler.

Constraints (also in CLAUDE.md):
- stdlib only, no third-party packages
- errors wrapped: fmt.Errorf("operation: %w", err)
- all JSON responses include a "status" field

Acceptance criteria:
- 200 + updated Task on success, CompletedAt set to time.Now()
- 404 + {"status":"error","error":"task not found"} if id not in store
- 409 + {"status":"error","error":"already complete"} if CompletedAt already set
- Route registered in router.go

Do not modify the store interface. Add a CompleteTask(id string) method if needed.
Do not create or modify test files.
```

[Cursor] Composer Setup

```
@tasks.go @router.go @memory.go @task.go

Add PATCH /tasks/:id/complete handler.

Constraints:
- stdlib only
- errors: fmt.Errorf("operation: %w", err)
- all JSON includes "status" field

Done:
- 200 + Task on success, CompletedAt = time.Now()
- 404 if not found, 409 if already complete
- Route registered in router.go
- Do not modify store interface
```

Note: Cursor's composer context is smaller. Keep the spec tighter. The `@file` references pull current file state — do not paste code.

Step 4 — After 3 Turns, Write a Checkpoint

Assume 3 turns have happened: - Turn 1: model read files, proposed the handler - Turn 2: you asked for a fix to the 409 condition - Turn 3: route was registered in router.go

Checkpoint Message

```
## Checkpoint — PATCH /tasks/:id/complete

### Verified state
- Handler implemented in internal/api/tasks.go (CompleteTask)
- Store method CompleteTask(id string) added to internal/store/memory.go
- Route registered: PATCH /tasks/{id}/complete in internal/api/router.go
- Task.CompletedAt field confirmed present in internal/models/task.go
- Manual test:
  curl -X PATCH localhost:8080/tasks/1/complete → 200 + Task JSON ✓
  curl -X PATCH localhost:8080/tasks/999/complete → 404 ✓
  curl -X PATCH localhost:8080/tasks/1/complete (again) → 409 ✓

### Active constraints
- stdlib only, no third-party packages
- errors: fmt.Errorf("operation: %w", err)
- JSON responses include "status" field
- Store interface: add methods only, do not modify existing signatures

### Files modified this session
- internal/api/tasks.go (added CompleteTask handler)
- internal/store/memory.go (added CompleteTask method)
- internal/api/router.go (registered PATCH route)

### Files NOT modified (confirmed unchanged)
- internal/models/task.go
- main.go

### Next action
Implement POST /tasks — see SPEC.md under "POST /tasks" for request/response spec
```

Step 5 — Evaluate the Checkpoint

Ask: can a fresh session pick up where this left off using only the checkpoint?

Evaluation criteria:

- [] Verified state is specific — says what was built and what was tested, not just “it works”
- [] Test results are concrete — actual curl commands with actual responses, not “tests pass”
- [] Constraints are fully restated — a fresh session doesn't need CLAUDE.md to be read to know the rules
- [] All modified files are listed — nothing to hunt for
- [] Files confirmed unchanged are listed — prevents re-applying work
- [] Next action is unambiguous — “see SPEC.md under POST /tasks” is better than “do the next endpoint”
- [] No reference to “what we discussed” — the checkpoint is self-contained

The test: paste only the checkpoint into a new session and ask the model to implement the next endpoint. If it asks clarifying questions that the checkpoint should have answered, the checkpoint is incomplete.

Reference: Checkpoint Template

```
## Checkpoint – [task name]

### Verified state
- [what was built, file-level]
- [manual test: command → result]

### Active constraints
- [list, restated fully]

### Files modified this session
- [file] ([what changed])

### Files NOT modified
- [file]

### Next action
[specific next task + pointer to spec]
```

Checklist

- ☐ Identified the three required context elements: files, constraints, done condition
 - ☐ Justified every included file with a direct reason
 - ☐ Justified every excluded file
 - ☐ Claude context message: files listed for reading, constraints stated, acceptance criteria specific
 - ☐ Cursor composer: @file references used, no pasted code, spec is tight
 - ☐ Checkpoint written after 3 turns, not after session ends
 - ☐ Checkpoint contains verified state with concrete test evidence
 - ☐ Checkpoint contains fully restated constraints (not “see CLAUDE.md”)
 - ☐ Checkpoint passes the fresh-session test: paste it and the session can continue
 - ☐ Next action in checkpoint is specific and points to a spec
-

Phase — 03-task-patterns

Directory: 03-task-patterns/

Source: 03-task-patterns/01-feature-pattern.md

Pattern: Implementing a New Feature

```
Implement <feature name> per <path/to/spec.md>.
Boundary: only touch files listed in the spec's boundary section.
Stop after implementation.
Do not write tests.
Do not add behavior not described in the spec.
```

Why the template is shaped this way

Spec on disk, referenced by path — not paraphrased. Paraphrasing introduces drift. The moment you summarize the spec in the prompt, you become the source of truth, and any gap in your summary becomes a gap in the implementation. The model reads the file; it does not rely on your memory of it.

“Stop after implementation.” Without this, the model will write tests. The problem: those tests are derived from the implementation, not the spec. A bug in the implementation produces a passing test. Tests must come from the spec — written separately, after the feature is in.

“Do not add behavior not described in the spec.” Models improve. They infer. They add “helpful” validations, extra fields, convenience methods. Every one of these is unreviewed scope. Reject them.

Boundary section in the spec. The spec must name which files may be created or modified. This is the blast radius. If a file appears in the diff that is not in the boundary list, the output is out of scope regardless of correctness.

Filled Example

Spec file: docs/specs/get-tasks.md

```
Boundary:
  create: internal/handler/task_handler.go
  modify: internal/router/router.go
  do not touch: store/, main.go, go.mod

GET /tasks
- Returns 200 with JSON array of all tasks
- Empty list returns 200 with []
- No filtering, no pagination
- Task shape: { id: string, title: string, done: bool }
```

Prompt sent to Claude Code or Cursor:

```
Implement GET /tasks per docs/specs/get-tasks.md.
Boundary: only touch files listed in the spec's boundary section.
Stop after implementation.
Do not write tests.
Do not add behavior not described in the spec.
```

Expected output: changes to task_handler.go and router.go only.

What to Reject

Signal	Why it's wrong
New file outside the boundary section	Out-of-scope change; reject regardless of quality
Test file created	Tests derived from code, not spec; delete and re-run from spec
Extra fields added to response struct	Unreviewed behavior; not in spec
Validation logic added that spec doesn't mention	Scope creep; remove
Comment: "I also updated X for consistency"	Not authorized; revert
go.mod or go.sum modified without being in boundary	Unauthorized dependency; reject

Checklist

- ☐ Spec file exists on disk before sending the prompt
- ☐ Spec has a boundary section listing exact file paths
- ☐ Prompt references the spec by path, not by description
- ☐ Prompt includes "stop after implementation"
- ☐ Prompt includes "do not write tests"
- ☐ Prompt includes "do not add behavior not in spec"
- ☐ After output: diff contains only files in the boundary section
- ☐ After output: no test files created
- ☐ After output: no behavior present that spec doesn't describe

Source: `03-task-patterns/02-bug-fix-pattern.md`

Pattern: Fixing a Bug

```
Bug: <observed behavior – exact, not interpreted>.
Expected: <correct behavior per spec or failing test>.
Hypothesis: <one sentence – specific code location and mechanism>.
Fix only the hypothesis.
Do not refactor unrelated code.
After the fix, verify with: <exact command>.
```

Why the template is shaped this way

You must state the hypothesis. Without it, the model treats the bug as open-ended. It searches for “improvements,” “potential issues,” “related problems.” The result is a diff that touches five things, one of which is the bug. You cannot tell which fix did the work. You cannot revert safely.

The hypothesis pins the fix to one mechanism. If the hypothesis is wrong, the fix is small and reversible. If it is right, the fix is small and verifiable.

Observed vs. expected — exact, not interpreted. “Returns wrong status” is interpreted. “Returns 404, expected 200” is exact. The model cannot drift on an exact description.

“Per spec or failing test.” The correct behavior must be sourced from a ground truth — not from your intuition. If a test already describes the correct behavior, reference it. If the spec does, reference it. “I expected it to return 200” is not a source. `internal/handler/task_handler_test.go:TestGetTask_found` is.

“Do not refactor unrelated code.” Explicit. Without this, the model will “clean up” surrounding code while fixing the bug. The cleanup is well-intentioned and hard to review. Reject it.

Verification command. Every fix needs a falsifiable check. If you cannot name the command before fixing, you do not understand the bug yet. The model cannot verify its own work without it — but it will confidently claim success anyway.

Filled Example

Bug: `POST /tasks` returns 404 when creating a new task. Expected: 201 with the created task body.

Hypothesis: the route is registered under `/task` (singular) in `router.go`, but the handler is mounted at `/tasks` (plural).

Prompt:

Bug: `POST /tasks` returns 404. Request body is valid JSON.

Expected: 201 with created task body, per `docs/specs/post-tasks.md`.

Hypothesis: route registered as `/task` (singular) in `internal/router/router.go`, handler mounted at `/tasks` (plural).

Fix only the hypothesis.

Do not refactor unrelated code.

After the fix, verify with: `go test ./internal/handler/... -run TestPostTask -v`

Expected diff: one line in `router.go` — the route path string.

What to Reject

Signal	Why it's wrong
Diff touches files not mentioned in the hypothesis	Fix is wider than the hypothesis; cannot verify scope
Handler logic rewritten “while we’re here”	Unauthorized refactor; revert
Test added that matches the (possibly wrong) implementation	Test derived from code, not spec
No verification command in the output	Unverifiable fix; ask for one before accepting
“I also noticed...” followed by additional changes	Additional changes are out of scope; separate ticket
Hypothesis changed in the output without explanation	Model decided your diagnosis was wrong; challenge it or confirm before accepting

Checklist

- ☐ Observed behavior is exact (specific status code, specific error message)
- ☐ Expected behavior is sourced from spec or failing test — not intuition
- ☐ Hypothesis names specific file and mechanism (not “somewhere in the handler”)
- ☐ Prompt includes “fix only the hypothesis”
- ☐ Prompt includes “do not refactor unrelated code”
- ☐ Verification command is specified before sending the prompt
- ☐ After output: diff is limited to what the hypothesis predicted
- ☐ After output: verification command passes
- ☐ After output: no test files modified to match new behavior

Pattern: Safe Refactoring

```
Refactor: <what is moving and where>.

CURRENT STATE:
<describe current structure – file, function, or package>

TARGET STATE:
<describe end structure – file, function, or package>

STEPS:
1. <atomic step – must compile and pass tests after this step alone>
2. <atomic step – must compile and pass tests after this step alone>
...

VALIDATION after each step: go build ./... && go test ./...

Do not combine steps.
Do not rewrite the file to be "cleaner."
Do not change behavior.
```

Why the template is shaped this way

The incremental rule. Each step must leave the codebase in a working state independently. No “we’ll fix the broken reference in step 3.” If step 2 breaks the build, step 2 is wrong — stop, diagnose, rewrite step 2. This rule makes refactors reviewable and revertable at any point.

CURRENT STATE / TARGET STATE. Refactoring is moving code, not rewriting it. Naming the current structure and the target structure eliminates ambiguity about what the model is authorized to change. Everything outside that scope is off-limits.

Validation after each step. `go build ./... && go test ./...` is the gate. Not “I think it compiles.” Not “the main logic looks right.” The gate is binary. Run it after each step before proceeding.

“Do not rewrite the file to be cleaner” — why this is explicitly forbidden. This is the most common refactor failure. The model sees messy code and cannot resist improving it. The “clean rewrite” has a different diff shape than a mechanical move. It is hard to review, hard to revert, and mixes two concerns. If you want to clean the code, that is a separate task — not part of this refactor.

Filled Example

Extracting store logic from `handler.go` into `store/store.go`.

Refactor: move store logic from internal/handler/handler.go into internal/store/store.go.

CURRENT STATE:

- internal/handler/handler.go contains an in-memory tasks map and CRUD operations on it.
- No store package exists.

TARGET STATE:

- internal/store/store.go contains a Store struct with Add, Get, List, Delete methods.
- internal/handler/handler.go depends on store.Store via interface; no direct map access.

STEPS:

1. Create internal/store/store.go with Store struct and methods copied verbatim from handler.go.
Validation: go build ./... && go test ./...
2. In handler.go, import store package; replace direct map access with store method calls.
Validation: go build ./... && go test ./...
3. Remove the in-memory map and CRUD operations from handler.go.
Validation: go build ./... && go test ./...

Do not combine steps.

Do not rewrite handler.go to be cleaner.

Do not change behavior.

What to Reject

Signal	Why it's wrong
Step that breaks go build before the next step is applied	Violates incremental rule; stop and fix the step
Step that changes function signatures “for clarity”	Behavior change disguised as refactor
“I rewrote handler.go to be cleaner while moving the logic”	Explicitly forbidden; revert and redo mechanically
Steps combined into one (“I moved and updated in the same step”)	Cannot validate intermediate state; reject
New error handling added that wasn't in original	Behavior change; separate task
Test updated to match new structure before validation passes	Test was the only thing making it pass; invalid

Checklist

- ☐ CURRENT STATE names exact file(s) and structure
- ☐ TARGET STATE names exact file(s) and structure
- ☐ Each step is atomic and independently compilable
- ☐ Validation command is specified (go build ./... && go test ./...)
- ☐ Prompt includes “do not combine steps”
- ☐ Prompt includes “do not rewrite the file to be cleaner”
- ☐ Prompt includes “do not change behavior”
- ☐ After each step: run the validation command before proceeding
- ☐ After final step: full test suite passes with no modifications to test files
- ☐ Diff shape matches “move” not “rewrite” — same logic in new location

Pattern: Architecture and Design Decisions

```
Given: <current state – what exists now, what constraints are fixed>.
Problem: <specific issue – why the current state is insufficient>.
Propose 2-3 options.
For each option:
  - Name
  - Implementation sketch (not full code – key types, key interactions)
  - Pros (concrete, not "simpler")
  - Cons (concrete, not "more complex")
  - Risks (what can go wrong at scale or under change)
  - What it forecloses (what future changes become harder or impossible)
Do not recommend.
I will decide.
```

Why the template is shaped this way

“Do not recommend.” When the model recommends, it anchors the decision. You read Option A (recommended) first, and Options B and C are evaluated against it. Your decision is no longer independent — it is a reaction. Architecture decisions have long tails; you need to own the choice without an anchor.

“I will decide.” Signals that the model’s job ends at analysis. Without this, the model often appends a recommendation anyway, framed as “summary” or “my take.” Explicit instruction stops it.

Implementation sketch, not full code. You need enough structure to evaluate feasibility — key types, key interactions, which packages or interfaces are involved. Full code at this stage means you are already committed to an option before deciding. The sketch reveals tradeoffs without locking you in.

“What it forecloses.” This is the most important field and the most often omitted. Every architecture decision eliminates future options. A mutex-based store forecloses concurrent streaming updates without a rewrite. A channel-based store forecloses synchronous batch operations without coordination overhead. These foreclosures are often invisible until you hit them.

“Concrete, not ‘simpler’.” “Simpler” is not a pro. “No lock contention under single-writer workload” is a pro. “Requires serializing all writes through a goroutine, adding latency to synchronous call paths” is a con. Force specificity.

Filled Example

Choosing a concurrency strategy for the task-api in-memory store.

```
Given:
- task-api is a Go HTTP service with an in-memory task store.
- Handlers run in separate goroutines per request (net/http default).
- Current store is a plain map with no synchronization – data races confirmed under load.

Problem:
- Concurrent reads and writes to the map cause data races.
- Need a synchronization strategy before adding more endpoints.

Propose 2-3 options.
For each option:
  - Name
  - Implementation sketch (key types, key interactions – not full code)
  - Pros (concrete)
  - Cons (concrete)
  - Risks (at scale or under change)
  - What it forecloses

Do not recommend.
I will decide.
```

Expected output: three options such as (1) `sync.RWMutex` on the store struct, (2) channel-based serialization with a goroutine owning the map, (3) `sync.Map`. Each with concrete sketch and filled fields.

What to Reject

Signal	Why it's wrong
Single option presented as “the best approach”	Anchors decision; no alternatives to evaluate
“I recommend Option 2” anywhere in the output	Violates “do not recommend”; request output without the recommendation
Pros/cons that say “simpler” or “more complex” without specifics	Not evaluable; ask for concrete rewording
No “what it forecloses” field	Missing the most important field; request completion
Full implementation code provided	Premature commitment; replace with a sketch
Options that are not genuinely different tradeoffs	e.g., mutex variant A vs. mutex variant B; ask for a structurally different option

Checklist

- ☐ GIVEN section describes current state with specific constraints (language, existing structure, fixed dependencies)
- ☐ PROBLEM is a specific issue, not a vague concern
- ☐ Prompt asks for 2-3 options — not 1, not 5
- ☐ Each option includes: name, sketch, pros, cons, risks, what it forecloses
- ☐ Prompt explicitly says “do not recommend”
- ☐ Prompt explicitly says “I will decide”
- ☐ After output: each option has a concrete implementation sketch
- ☐ After output: pros and cons are specific, not adjective-based
- ☐ After output: “what it forecloses” is filled for each option
- ☐ Decision made independently, recorded in an ADR or spec before implementation begins

Source: `03-task-patterns/05-review-pattern.md`

Pattern: Code Review

[Claude] Reference file by path or use `@filename` in Claude Code. [Cursor] Use `@filename` to attach the file in Cursor chat.

```
Review <file path or diff>.
Classify each finding:
  CRITICAL — breaks correctness (wrong output, data loss, race condition)
  HIGH      — security or data integrity issue
  MEDIUM   — maintainability problem (unclear ownership, hidden coupling, missing error handling)
  LOW       — style, naming, minor inconsistency

For each finding:
  - Location: <file>:<line> or function name
  - What is wrong: <specific description>
  - What correct looks like: <concrete alternative — pseudocode or description>

Do not fix.
Report only.
```

Why the template is shaped this way

“Do not fix. Report only.” Fixing during review mixes two concerns. The review output is a document — it must be readable, diffable, and discussable without also being a patch. When a model fixes while reviewing, the findings and the fixes blur together.

You cannot accept a finding and reject its fix. You cannot share the review with a teammate. The separation is structural, not stylistic.

Severity classification. Without classification, every finding lands at the same weight. “This variable name is confusing” gets equal attention as “this write is not protected by the mutex.” Classification forces triage into the output — you apply HIGH+CRITICAL immediately, you decide on MEDIUM, you defer LOW.

Location as `file:line` or function name. “In the handler function” is not a location. `internal/handler/task_handler.go:47` is. If a finding cannot be located exactly, it cannot be acted on. Vague location signals a vague finding — ask for specificity.

“What correct looks like.” Without this field, you have a complaint, not a review. “This is not thread-safe” is a complaint. “This should use `mu.Lock()` / `mu.Unlock()` around the map write at line 52” is a finding. The alternative does not have to be complete code — pseudocode or a description of the correct pattern is enough.

Filled Example

[Claude]

```
Review internal/handler/task_handler.go.
Classify each finding: CRITICAL, HIGH, MEDIUM, or LOW.
For each finding:
  - Location: file:line or function name
  - What is wrong: specific description
  - What correct looks like: concrete alternative
Do not fix.
Report only.
```

[Cursor]

```
Review @task_handler.go.
Classify each finding: CRITICAL, HIGH, MEDIUM, or LOW.
For each finding:
  - Location: file:line or function name
  - What is wrong: specific description
  - What correct looks like: concrete alternative
Do not fix.
Report only.
```

Expected output shape:

```
CRITICAL — internal/handler/task_handler.go:52
What is wrong: tasks map written without lock; concurrent requests cause a data race.
What correct looks like: acquire mu.Lock() before write, defer mu.Unlock().

HIGH — CreateTask():88
What is wrong: task ID generated from math/rand without seed; collisions likely under load.
What correct looks like: use crypto/rand or google/uuid for ID generation.

MEDIUM — ListTasks():31
What is wrong: returns nil on empty store; callers must nil-check or panic.
What correct looks like: return empty slice ([]Task{}) for consistent JSON serialization.

LOW — task_handler.go:12
What is wrong: package-level var for store makes dependency implicit.
What correct looks like: inject store via constructor or handler struct.
```

How to Use the Output

After receiving the review:

1. **CRITICAL** — fix before any further development. Non-negotiable.
2. **HIGH** — fix before merging. Security and data integrity issues compound.

- 3. **MEDIUM** — decide: fix now or create a tracked issue. Do not silently defer.
- 4. **LOW** — decide: batch into a style pass or defer. Do not let LOW findings block shipping.

When fixing, use the bug-fix pattern (02) for each CRITICAL/HIGH finding — one fix per finding, hypothesis stated, verification command named.

What to Reject

Signal	Why it’s wrong
Finding without severity label	Cannot triage; ask for classification
“This could be improved” without specifics	Not a finding; ask for location and what correct looks like
Finding that includes a code fix inline	Mixes review and implementation; request report-only version
All findings are LOW or MEDIUM when a race condition is present	Under-severity; push back on classification
“The code looks good overall” with no findings	Not a review; send with explicit instruction to look for specific concerns
Findings numbered but unsorted — CRITICAL buried after 12 LOWs	Ask for output sorted by severity descending

Checklist

- ☐ File or diff is referenced by path, not pasted as text in the prompt
- ☐ Prompt specifies all four severity levels with their definitions
- ☐ Prompt specifies the three fields for each finding: location, what is wrong, what correct looks like
- ☐ Prompt includes “do not fix”
- ☐ Prompt includes “report only”
- ☐ After output: every finding has a severity label
- ☐ After output: every finding has an exact location
- ☐ After output: every finding has a concrete alternative (not a complaint)
- ☐ After output: CRITICAL and HIGH findings addressed before proceeding
- ☐ MEDIUM findings either fixed or tracked; LOW findings explicitly deferred

Source: 03-task-patterns/06-lab-apply-the-right-pattern.md

Lab: Apply the Right Pattern

No prompts are given to you. You select the pattern, write the prompt, apply it, and audit the output.

Setup

This lab assumes the task-api project from earlier in the path. If you do not have it, use any small HTTP service in your language. The patterns apply regardless of language; the verification commands change.

Four Tasks

Read each task. Do not write a prompt yet.

- Task A — New endpoint** The spec at docs/specs/delete-task.md describes DELETE /tasks/{id}. The spec includes a boundary section. Your job is to implement it.
- Task B — Broken response** GET /tasks/{id} returns a 404 for a task ID that exists in the store. A test TestGetTask_found in internal/handler/task_handler_test.go describes the correct behavior (200 with task body). The test is failing.
- Task C — Extract logic** Input validation (checking that title is non-empty, that id is a valid UUID) lives inside CreateTask() in handler.go. It needs to move to internal/validator/validator.go without changing behavior.

Task D — Storage decision The current in-memory store uses a plain map with a `sync.RWMutex`. You are about to add a streaming endpoint (`GET /tasks/stream`) that pushes SSE events when tasks change. You need to decide whether to keep the mutex-based store, switch to a channel-based store, or introduce a third approach.

Step 1: Pattern Selection

For each task, identify the correct pattern from this directory. Write your answer and a one-sentence justification.

Task	Pattern file	Justification
A		
B		
C		
D		

Expected answers (check after completing on your own):

Task	Pattern	Why
A	01-feature-pattern.md	Spec exists on disk; boundary section present; implementation only
B	02-bug-fix-pattern.md	Specific observed vs. expected behavior; existing failing test as ground truth
C	03-refactor-pattern.md	Moving code without changing behavior; must be atomic and incremental
D	04-architecture-pattern.md	Decision between structural alternatives; no implementation yet

Step 2: Write Two Prompts

Choose **two** of the four tasks. Write the full prompt for each using the pattern template.

Requirements: - Copy the template from the pattern file. - Fill every field — no placeholders. - For Task A: spec file must exist before you write the prompt. Create a minimal spec file if needed. - For Task B: state the hypothesis as one sentence with file and mechanism. - For Task C: write all steps; each must be independently compilable. - For Task D: include all six fields per option; end with “do not recommend.”

Write your prompts here before sending them:

Prompt for Task ____:

(your prompt here)

Prompt for Task ____:

(your prompt here)

Step 3: Send and Observe

Send each prompt to Claude Code or Cursor. Do not intervene mid-output.
After each response, run the “What to Reject” checklist from the relevant pattern file.

Step 4: Audit the Output

Record your observations for each task you prompted.

Task ____:

Check	Pass / Fail	Notes
Output stayed within stated scope		
No unauthorized files modified		
No behavior added beyond the prompt		
Verification command (if applicable) passes		
Specific rejection trigger (if any)		

Task ____:

Check	Pass / Fail	Notes
Output stayed within stated scope		
No unauthorized files modified		
No behavior added beyond the prompt		
Verification command (if applicable) passes		
Specific rejection trigger (if any)		

Step 5: Record What Was Rejected

For each output, list what you rejected and why.

If nothing was rejected, write “nothing rejected” — then ask: was the output genuinely clean, or did you not look hard enough? Apply the checklist again.

Task ____: - Rejected: - Why:

Task ____: - Rejected: - Why:

Lab Completion Checklist

- ☐ Pattern selected for all four tasks with justification
- ☐ Two full prompts written before sending (no placeholders)
- ☐ Output audited against the “What to Reject” checklist for both tasks
- ☐ Rejected items recorded with specific reasons
- ☐ Verification command ran and passed (for Tasks A, B, C)
- ☐ No fixes applied during a review-only output (if Task B review was involved)
- ☐ One sentence written: what would break if you had used the wrong pattern for one of your tasks

Phase — 04-failure-patterns

Directory: 04-failure-patterns/

Source: 04-failure-patterns/01-ambiguity-failure.md

Ambiguity Failure

Detection signals

- Output is coherent, compiles, passes a surface read — but answers a different question than you meant
- The implementation is internally consistent with *an* interpretation of your goal, just not yours
- No error; the AI was not wrong — it was *precise about the wrong thing*
- You find yourself saying “that’s not what I meant” rather than “that doesn’t work”

Why it happens

A goal statement that permits two distinct implementations is ambiguous. The model picks one — consistently and confidently — which makes the failure hard to spot until you run it against real requirements.

Classic ambiguous statement: “**make tasks searchable**”

Valid interpretations: - Search by title, case-insensitive substring match - Search by numeric task ID, exact match - Full-text search across title + description - Filter by tag or status (also “searching” by some definitions)

The model commits to one of these. The code it produces is correct for that interpretation.

Examples: ambiguous vs unambiguous

Ambiguous	Unambiguous
make tasks searchable	add GET /tasks?q= that returns tasks whose title contains q as a case-insensitive substring
improve error handling	return 400 with JSON {"error": "..."} for all validation failures; never return 500 for input errors
cache the results	cache the response of GET /tasks in memory for 60 seconds; invalidate on any POST /tasks
make the API faster	reduce p99 latency of GET /tasks below 50ms; do not change the response shape
add authentication	add Bearer token middleware that rejects requests without a valid token with 401; no login endpoint yet

Recovery procedure

Do not ask “why did you do X?” — that produces a justification for the wrong implementation.

1. Stop. Do not build on the wrong output.
2. Restate the goal as a **single sentence** with no OR-paths.
3. Add one concrete example of correct output (a sample request + expected response, or a test case).
4. Re-prompt from the last known-good state.

```
# Recovery prompt structure
```

```
Task: [single unambiguous sentence]
```

```
Example of correct output:
```

```
Input: GET /tasks?q=deploy
```

```
Output: 200 {"tasks": [{"id": 3, "title": "Deploy to staging"}]}  
(partial match, case-insensitive, title only)
```

```
Implement this. Do not add search to any other field.
```

Prevention

Before submitting any goal statement, ask:

```
"What are the two most different ways to implement this?"
```

If two valid implementations exist, the goal is ambiguous. Collapse the ambiguity before prompting.

Apply this to the four most common ambiguity sources:

1. **Scope** — which fields, which endpoints, which layers?
2. **Match semantics** — exact, prefix, substring, fuzzy?
3. **Error behavior** — what happens when input is missing, malformed, out of range?
4. **Side effects** — does this operation mutate state, trigger events, invalidate cache?

Write the constraint on each dimension before writing the goal.

```
# Prevention template
```

```
Goal: [verb] [noun] [scope]
```

```
Match: [exact | prefix | substring | none applicable]
```

```
On error: [return X | panic | ignore]
```

```
Side effects: [none | invalidate Y | emit event Z]
```

Lab: ambiguity audit

Take any prompt you've written this week. Run the two-interpretation test.

For each prompt, write: - Interpretation A (what you meant) - Interpretation B (what a reasonable model might also do)

If you cannot construct Interpretation B, the prompt is unambiguous. If you can, rewrite before running.

Checklist

- ☐ Goal restated as a single sentence with no OR-paths
 - ☐ At least one concrete example of correct output included
 - ☐ Scope constrained: which fields / endpoints / modules
 - ☐ Match semantics explicit when any search/filter/lookup is involved
 - ☐ Error behavior specified
 - ☐ Two-interpretation test applied and passed
 - ☐ Not building on output until interpretation is confirmed correct
-

Hallucination Recovery

Detection signals

- A function or method name appears in the output that does not exist in your codebase
 - Behavior is described that no test covers and no code implements
 - The model refers to a prior conversation or decision that never happened (“as we discussed”, “based on what you said earlier”)
 - An import path is slightly wrong — package exists but the specific symbol does not
 - The model describes what a function *should* do, not what it *does*
 - A struct field is used that isn't in your type definition
-

Why it happens

The model generates plausible continuations. When it lacks a ground truth (your actual codebase, the actual API), it fills gaps with statistically likely names and behaviors. The output is syntactically valid and semantically coherent. It is not true.

Hallucinations compound: if you accept an invented function name and continue, the next turn builds on that invention. By turn 4 you have a coherent but entirely fictional subsystem.

Do not continue the session after detecting a hallucination.

Recovery: 3 steps

1. **Identify exactly what was invented.** Name it specifically. “The function `task.FindByTag` does not exist” — not “something seems wrong.”
2. **Isolate it. Do not merge or accept the output.** In Claude Code: do not run `git add`. In Cursor: reject the diff. Treat the entire output as suspect if it contains one hallucination.
3. **Re-prompt with the specific correction and the real source.** Give the model the actual thing to use, not just a correction.

```
# Recovery prompt structure
```

```
The function `task.FindByTag` does not exist in this codebase.  
The real function is `task.ListByFilter(ctx, filter TaskFilter)` – see internal/task/query.go line 44.  
  
Rewrite the handler using `task.ListByFilter`. Do not invent any other function names.  
Before using any function, confirm it exists in the file I just referenced.
```

The grounding instruction

Add this to any prompt where the model will write code against your codebase:

```
Verify each function name exists before using it.  
If you are unsure whether a function exists, stop and ask – do not invent a plausible name.
```

This works because it shifts the model from generation mode (“what would make sense here?”) to verification mode (“does this actually exist?”).

For larger codebases, be more specific:

```
All task operations are in internal/task/.  
All HTTP handlers are in internal/api/.  
Do not use any function from a package not listed here without asking first.
```

Tool-specific notes

Claude Code: Hallucinations in code are caught by `go build`. Run the build after every non-trivial generation. A build failure on a function that “should exist” is a hallucination signal — stop there, do not try to fix the invented function, fix the prompt.

```
go build ./... 2>&1 | grep "undefined:"
```

Undefined symbols are the hallucination ledger. Each one is an invented name.

Cursor: The editor will not stop you from accepting a diff that calls a nonexistent function. Check every new function name before accepting: - Hover to see if the IDE can resolve it - Search the codebase for the exact symbol - If the IDE shows an error after accept, reject immediately — do not try to “fix” an invented API

Prevention

```
# Grounding block – add to any code-writing prompt
```

Context:

- Task model: `internal/task/model.go`
- Task queries: `internal/task/query.go`
- HTTP handlers: `internal/api/task_handler.go`

Rules:

- Use only functions that exist in the files above
- If you need a function that doesn't exist, say so — do not invent one
- After writing code, list every function you called and which file it comes from

The self-audit at the end (“list every function you called”) catches hallucinations before you do. The model will either correctly cite the file or flag that it is uncertain.

Checklist

- [] Build run after generation; zero undefined symbols
 - [] Every new function name verified to exist in the codebase
 - [] No output accepted that references a prior conversation you cannot confirm
 - [] Grounding instruction in any prompt that touches existing code
 - [] Session not continued after detecting one hallucination
 - [] Isolation confirmed: hallucinated output not staged or merged
 - [] Recovery prompt includes the real function/file as the explicit replacement
-

Source: `04-failure-patterns/03-constraint-failure.md`

Constraint Failure

Detection signals

- A package you prohibited appears in an import block
 - A file you said not to touch has a diff
 - A dependency was added when you said `stdlib-only`
 - The model added an abstraction layer you explicitly excluded
 - Output is correct for the happy path but the constraint applied to edge cases is ignored
 - The constraint is present in the previous prompt; it is absent from the behavior
-

Why it happens

Three causes, in order of frequency:

- 1. **Soft language.** “Prefer”, “try to”, “where possible”, “ideally”, “avoid if you can” — these are not constraints. They are suggestions the model weighs against other factors and may override.
- 2. **Buried rule.** A constraint in the middle of a 12-line prompt competes with everything around it. The model’s attention is not uniform. Rules at position 6 of 10 lose.
- 3. **Implicit assumption conflict.** The model has a strong prior about how to solve this class of problem. “Add search to the task API” carries an implicit assumption that you’ll use a query library. Your constraint (“stdlib only”) conflicts with that prior. The prior wins unless the constraint is stated louder.

Three reinforcement techniques

1. Move the constraint to the first line

```
# Weak (buried)

Add a search endpoint to the task API.
It should return tasks matching a query string.
Keep it simple and readable.
Do not import any external packages – use stdlib only.
Add tests.

# Strong (first line)

CONSTRAINT: stdlib only – no external packages.

Add a GET /tasks?q= endpoint that returns tasks whose title contains q
as a case-insensitive substring. Use strings.Contains. Add tests.
```

2. Make it binary (must / must not)

Replace all soft qualifiers with hard ones:

Soft	Hard
prefer stdlib	must use stdlib only
try not to touch other files	must not modify any file outside internal/task/
avoid global state	must not use any package-level variables
keep it simple	must not introduce any new interfaces

Soft language gives the model a decision to make. Binary language removes the decision.

3. Add a verification step

After stating the constraint, add an explicit self-audit:

```
After implementing, list every external package you imported.
The list must be empty.
If it is not empty, stop and revise before showing me the code.
```

This is the most reliable technique. It forces the model to check its own output against the constraint before you see it. Models catch their own violations when asked to audit explicitly.

When the rule is violated

Do not continue. Do not try to patch the violation inline.

- 1. **Revert the change.** In Claude Code: `git checkout -- .` In Cursor: reject the diff.

2. **Tighten the constraint.** Binary + first-line + verification step.

3. **Restart with the hardened prompt.**

Building on a constraint violation produces output that assumes the violation is acceptable. The next turn digs the hole deeper.

Examples

Example 1: stdlib-only violated

Before (failed)

Add an in-memory cache for GET /tasks results.
Cache should expire after 60 seconds.
Use stdlib only.

After (hardened)

CONSTRAINT: must use stdlib only – must not import any package outside the Go standard library. If you need a cache, implement it with sync.Map and time.Now(). Do not use github.com/patrickmn/go-cache or any other external package.

Add an in-memory cache for GET /tasks.
After implementing, list every import. The list must contain only stdlib packages.

Example 2: file boundary violated

Before (failed)

Refactor the task search logic to be more testable.

After (hardened)

CONSTRAINT: must not modify any file outside internal/task/query.go.
Do not touch handlers, models, or main.go.

Extract the filter logic in internal/task/query.go into a pure function that takes a []Task and a string query and returns []Task.
After implementing, list every file you modified. The list must contain only internal/task/query.go.

Prevention

Append this block to any prompt that has a hard constraint:

Constraints (non-negotiable):

- [constraint 1 as must/must not statement]
- [constraint 2 as must/must not statement]

Verification: after implementing, confirm each constraint above was followed.
List any constraint you could not follow and why.

The verification step creates an accountability checkpoint. If the model cannot follow a constraint, it will say so before you discover it in review — which is the outcome you want.

Checklist

- [] Every constraint stated as must/must not — no soft qualifiers
- [] Constraint appears on the first line or in a clearly marked block

- [] Verification step added: model asked to audit its own output
- [] On violation: change reverted before continuing
- [] Tightened prompt used on restart, not the original
- [] Binary constraint confirmed in generated import list or file diff

Source: 04-failure-patterns/04-context-drift.md

Context Drift

Detection signals

- Turn 3 output contradicts a constraint that turn 1 output correctly followed
- The model produces something it already produced and you already rejected
- A decision you made three turns ago is reversed without you changing the prompt
- The model's behavior gradually shifts across a session without any prompt change
- You re-read your earlier prompt and confirm the rule is there — the model simply stopped following it

Why it happens

In a multi-turn session, the model does not have perfect recall of every earlier message. After several turns, early messages are compressed into a summary representation. The constraint is still “there” in some sense, but it no longer has the weight or precision of the original statement.

The model is not ignoring you. It is working from a degraded version of what you said.

This is not a bug. It is how attention over long contexts works. The operating cost of treating every token in a long session equally is prohibitive. Early tokens get less weight.

The constraint you stated once in turn 1 is not the constraint you have in turn 6.

Recovery

Do not repeat the constraint in the same turn where you detect the drift. That does not fix the underlying issue — it adds one more instance of the constraint to an already degraded context.

Restart the session with a checkpoint.

1. Stop the current session.
2. Write a checkpoint prompt that includes:
3. The current state of the work (what was built, what was decided)
4. The non-negotiable constraints stated explicitly, not by reference
5. The specific next task
6. Open a new session (new conversation in Claude Code; new composer in Cursor) and start from the checkpoint.

```
# Checkpoint prompt structure

## Current state
- GET /tasks endpoint exists in internal/api/task_handler.go
- Task model defined in internal/task/model.go
- No external packages imported; all stdlib

## Non-negotiable constraints
- Must use stdlib only – no external packages
- Must not modify internal/api/task_handler.go
- All new code goes in internal/task/query.go

## Next task
Add substring search: a function ListByTitle(tasks []Task, q string) []Task
that returns tasks whose Title contains q (case-insensitive).
```

This is more work than re-reading the original prompt. It is less work than debugging context drift across four more turns.

Prevention

Write the most important constraint in every turn where it applies — not just the first.

The repetition feels redundant. It is not. You are not reminding yourself. You are providing the constraint at full weight in every turn’s working context.

The constraint anchor

A 2-3 line block appended to every prompt in a session:

```
---
Anchor: stdlib only | no external packages | changes in internal/task/ only
---
```

Keep it short. Keep it identical across turns. Paste it at the end of every prompt.

The anchor serves two functions: 1. It ensures the constraint appears at full weight in each turn’s local context, regardless of how early turns have been compressed. 2. It makes drift visible: if the model violates a constraint that appeared in the same turn’s prompt, that is a different failure (constraint failure, not context drift).

Distinguishing context drift from constraint failure

Constraint failure	Context drift
Constraint in prompt; model ignores it	Constraint from earlier turn; model forgets it
Soft language or buried rule	Hard rule, correctly followed in early turns
Fix: harden the constraint in same session	Fix: restart with checkpoint
Happens on turn 1 as easily as turn 5	Happens more as session length grows

If you added the anchor and the model still violated a constraint stated in the same turn — that is constraint failure, not drift. Apply the constraint failure recovery.

Session length heuristic

Context drift becomes likely when: - The session is longer than ~15 substantive turns - You are deep in a debugging loop (many small turns) - You have introduced major new context mid-session (pasted a new file, changed the goal)

At that point, proactively write a checkpoint rather than waiting to detect drift.

Checklist

- [] Constraint anchor appended to every prompt in sessions longer than 5 turns
- [] On detecting drift: session stopped, not patched inline
- [] Checkpoint includes current state + explicit constraints + next task
- [] New session started from checkpoint, not continuation of drifted session
- [] Anchor is identical across turns (variation introduces its own ambiguity)
- [] Session restart triggered proactively at 15+ turns, not just reactively

Source: 04-failure-patterns/05-lab-diagnose-a-failing-prompt.md

Lab: Diagnose a Failing Prompt

Four broken prompt+output pairs from a Go task API project. For each: 1. Identify the failure mode 2. Write the diagnosis 3. For cases 2 and 4: write the recovery prompt 4. Apply the prevention technique for all four

Setup: the codebase context

```
task-api/  
  internal/  
    task/  
      model.go      - type Task struct { ID int; Title string; Done bool }  
      query.go       - func List(ctx context.Context) ([]Task, error)  
      store.go       - SQLite-backed persistence  
    api/  
      task_handler.go - HTTP handlers, uses task.List  
  main.go  
  go.mod             - module task-api, no external dependencies
```

Case 1

Prompt:

```
Add filtering to the task list endpoint.
```

Output: The model added `GET /tasks?done=true` to filter by completion status. It implemented `done=true` and `done=false` query params with correct SQL.

Your intent: you wanted to filter by keyword in the title.

Diagnosis format

```
Failure mode: [ambiguity | hallucination | constraint failure | context drift]  
What was invented or misread: [specific statement]  
Why the model's behavior was reasonable: [explain the model's valid interpretation]  
What broke: [what you got vs what you needed]
```

Fill this in before reading further.

Reference diagnosis

```
Failure mode: ambiguity  
What was misread: "filtering" permits filtering by status, title, tag, date, or any field  
Why the model's behavior was reasonable: filtering by done/undone is the most common task-list  
  filter pattern; the model produced a fully correct implementation of a valid interpretation  
What broke: the endpoint filters by completion status; you needed title substring search
```

Case 2

Prompt:

```
Refactor task.List to support a filter argument so the handler can pass search criteria.
```

Output (turn 1): Model added `func List(ctx context.Context, f Filter) ([]Task, error)` where `Filter` is a new struct. Handler updated correctly. Build passes.

Turn 2 prompt:

```
Now add the title search - filter by substring match on Title.
```

Turn 2 output:

```
import "github.com/jmoiron/sqlx"

func (s *Store) Search(ctx context.Context, q string) ([]Task, error) {
    return sqlx.SelectContext(ctx, s.db, &results,
        "SELECT * FROM tasks WHERE title LIKE ?", "%"+q+"%")
}
```

sqlx is not in go.mod. The Store type has no db field. sqlx.SelectContext does not exist with this signature.

Your tasks for Case 2

2a. Diagnosis:

Failure mode:
 What was invented:
 Why the model's behavior was reasonable:
 What broke:

2b. Recovery prompt — write a prompt that will produce correct output. Include: - Identification of what was invented - The real types/functions to use (from the codebase description above) - The grounding instruction

Reference diagnosis

Failure mode: hallucination
 What was invented: sqlx package (not in go.mod), Store.db field (doesn't exist in store.go), sqlx.SelectContext signature (invented API)
 Why reasonable: sqlx is a common Go database package; the model generated a statistically plausible database layer
 What broke: code does not compile; imports a nonexistent package; calls a nonexistent method

Reference recovery prompt

Do not use sqlx — it is not in go.mod and must not be added.

The real store is in internal/task/store.go. It uses database/sql directly. The Store struct has a field `db \*sql.DB`. Use db.QueryContext for queries.

Add a search method to Store that accepts a query string and returns []Task filtered by case-insensitive substring match on Title. Use:

```
db.QueryContext(ctx, "SELECT id, title, done FROM tasks WHERE title LIKE ?", "%"+q+"%")
```

Verify: list every import you use. All must be stdlib. List every method you call on Store.db — all must be methods on *sql.DB.

Case 3

Session turn 1 prompt:

CONSTRAINT: must not modify task_handler.go.

Add a ListByTitle function to internal/task/query.go that filters tasks by title substring. Return []Task. No new packages.

Turn 1 output: Correct. query.go updated. task_handler.go untouched. Build passes.

Turn 6 prompt (after 4 turns of debugging an unrelated test):

Wire the ListByTitle function into the search endpoint.

Turn 6 output: Model modifies task_handler.go to add the search route, and also modifies main.go to register it.

Diagnosis format

```
Failure mode:
What changed between turn 1 (followed constraint) and turn 6 (violated it):
Evidence that this is not constraint failure:
What broke:
```

Fill this in before reading further.

Reference diagnosis

```
Failure mode: context drift
What changed: after 4 intervening turns, the early constraint on task_handler.go has been
  compressed; "wire it in" carries an implicit assumption that the handler needs to change
Evidence this is not constraint failure: the constraint was binary and first-line; the model
  followed it correctly in turn 1 when it was at full weight
What broke: task_handler.go was modified in violation of the constraint; main.go was also
  touched (never constrained, but unexpected)
```

Case 4

Prompt:

```
The task list endpoint is slow. Make it faster.
```

Output: Model added an in-memory LRU cache using `github.com/hashicorp/golang-lru/v2`. Rewrote the handler to check the cache before querying the store. Cache TTL logic added.

go.mod was not in the prompt context. There is no existing caching infrastructure. The model assumed an external package was acceptable and that LRU was the right strategy.

Your tasks for Case 4

4a. Diagnosis:

```
Failure mode: (there are two here – name both)
Constraint violated (implicit):
What was assumed without being stated:
What broke:
```

4b. Recovery prompt — write a prompt that will produce a correct, stdlib-only in-memory cache for the task list.

Reference diagnosis

```
Failure mode: ambiguity + constraint failure (implicit constraint)
Ambiguity: "make it faster" has many valid implementations – caching, query optimization,
  index hints, pagination, response compression
Constraint violated: stdlib-only is implicit in a project with no external deps; the model
  did not know this without being told
What was assumed: LRU is the right cache strategy; external packages are acceptable
What broke: external package imported; cache strategy chosen without input; handler
  rewritten without scoped instruction
```

Reference recovery prompt

CONSTRAINT: stdlib only — must not import any package outside the Go standard library.
CONSTRAINT: changes must be limited to internal/task/query.go and internal/api/task_handler.go.

Performance goal: cache the result of task.List in memory for 30 seconds.
Use sync.Map to store the cache entry. Use time.Now() to track expiry.
Invalidation: clear the cache on any successful task creation or deletion.

Do not use github.com/hashicorp/golang-lru or any other external cache library.

After implementing, list every import across all modified files.
All imports must be stdlib. If any are not, revise before showing me the code.

Prevention application

For each case, write the prevention technique you would apply to the *next* task of the same type.

Case	Prevention technique	What you'd add
1	Two-interpretation test	Before prompting: "what are the two most different ways to implement filtering?"
2	Grounding instruction	Add to prompt: "verify each function exists before using it; if unsure, ask"
3	Constraint anchor	Append to every turn: Anchor: must not modify task_handler.go
4	Binary constraint + verification step	First line: CONSTRAINT: stdlib only ; end: "list every import — must be stdlib"

Checklist

- ☐ All four failure modes identified correctly
- ☐ Case 2 diagnosis names specific invented symbols
- ☐ Case 2 recovery prompt includes real types/functions as replacements
- ☐ Case 3 diagnosis distinguishes context drift from constraint failure
- ☐ Case 4 diagnosis names both failure modes
- ☐ Case 4 recovery prompt includes binary constraint + verification step
- ☐ Prevention technique written for all four cases
- ☐ Each prevention technique matches the specific failure mode (not generic)

Phase — 05-workspace-configuration

Directory: 05-workspace-configuration/

Source: 05-workspace-configuration/01-claude-md-vs-cursorrules.md

CLAUDE.md vs .cursorrules

Both files load automatically when a session starts. The developer does not type project context, stack details, or standing constraints into the first message. The model already has them.

What each file does

CLAUDE.md [Claude] - Loaded from project root and merged with `~/ .claude/CLAUDE.md` (global) - Markdown format; supports headers, lists, code blocks - Read before the first tool call in every session - Applies to all Claude Code interactions in that working tree

.cursorrules [Cursor] - Loaded from project root only - Plain text or markdown (Cursor strips formatting for injection) - Applied to every Composer and Chat session automatically - No global equivalent — Cursor uses User Rules in Settings for that

What goes in both

```
# Project
task-api — REST API for task management
Stack: Node.js 20, Express, PostgreSQL, Jest

# Key paths
src/handlers/    — route handlers
src/services/    — business logic
src/db/          — query modules
tests/unit/      — Jest unit tests
tests/integration/ — supertest integration tests

# Constraints
- Standard library and declared dependencies only. No new packages without explicit approval.
- No global mutable state. All state flows through function parameters.
- Every handler must have a corresponding spec file before implementation begins.

# Code style
- Functions over classes for stateless logic
- Named exports only (no default exports)
- Error messages in the format: { error: string, code: string }
```

Paste that content into CLAUDE.md and a nearly identical block into .cursorrules. Both tools will enforce it from session start.

What CLAUDE.md has that .cursorrules doesn't

@import — split rules across files to keep CLAUDE.md maintainable:

```
# CLAUDE.md
@import .claude/rules/api-constraints.md
@import .claude/rules/testing-standards.md
@import .claude/rules/database-rules.md
```

Each imported file is a focused rule set. The root CLAUDE.md stays short.

Tool permissions — declare what Claude Code may and may not run:

```
## Permissions
- Allowed: read any file, run tests, run lint
- Require approval: git push, database migrations, npm install
- Never run: rm -rf, DROP TABLE, any production credentials
```

Hook configuration — run scripts on session events:

```
## Hooks
- Before commit: run npm test && npm run lint
- On file write to src/: update the relevant test file if it exists
```

.cursorrules has none of these. It is instruction text only.

What .cursorrules has that CLAUDE.md doesn't

Simpler syntax — no structure required. Cursor reads it as a flat prompt prefix.

Natural @file integration — Cursor's @file references in chat work alongside .cursorrules without extra configuration. Rules and file context compose cleanly.

Rule of thumb

If you would type it in the first message of every session, it belongs in workspace config.

"Remember, we're using stdlib only — no lodash, no moment"	→ workspace config
"This project uses PostgreSQL not MySQL"	→ workspace config
"Explain how async/await works"	→ prompt, not config
"Help me think through this architecture"	→ prompt, not config

Side-by-side: the same rule in both formats

Constraint: No global mutable state

```
<!-- CLAUDE.md [Claude] -->
## Constraints
- No global mutable state. All state must flow through function parameters or
  be encapsulated in a returned object. Module-level constants (frozen) are allowed.
  If asked to introduce global state, flag this as a constraint violation and suggest
  an alternative before proceeding.
```

```
# .cursorrules [Cursor]
CONSTRAINT: No global mutable state.
All state flows through function parameters or returned objects.
Module-level frozen constants are allowed.
If I ask you to introduce global state, tell me this violates the project constraint
and suggest a parameter-passing alternative before writing any code.
```

The behavior expected is identical. The format differs: CLAUDE.md uses structured Markdown; .cursorrules uses plain imperative text.

Side-by-side: stdlib-only constraint

```
<!-- CLAUDE.md [Claude] -->
## Dependencies
- Use only Node.js standard library and packages already listed in package.json.
- Do not suggest or introduce new npm packages.
- If a task seems to require a new package, name the stdlib alternative or ask for
  explicit approval before proceeding.

# .cursorrules [Cursor]
DEPENDENCIES: Use only Node.js stdlib and packages in package.json.
Do not add new npm packages.
If a task seems to need a new package, propose the stdlib approach first.
Ask for explicit approval before suggesting an install.
```

Checklist

- [] CLAUDE.md exists at project root with project name, stack, key paths, and constraints
- [] ~/.claude/CLAUDE.md has global rules that apply across all projects
- [] .cursorrules exists at project root with the same core constraints in plain text
- [] CLAUDE.md uses @import if rule count exceeds ~30 lines
- [] Tool permissions are declared in CLAUDE.md if any commands need to be restricted
- [] Every rule passes the “would I say this every session?” test
- [] No rule is in CLAUDE.md only because of formatting preference — if .cursorrules supports it, it’s in both

Source: 05-workspace-configuration/02-rules-that-work.md

Rules That Work

What makes a rule work

A rule works when it has three properties:

Binary — the model either followed it or didn’t. No partial credit. **Specific** — names the exact behavior, not a direction of preference. **Testable** — you can write a prompt that would violate it, run it, and check.

A rule that lacks any one of these is not a rule. It is a suggestion, and models treat suggestions as optional.

What makes a rule fail

These words kill rules:

Broken phrasing	Why it fails
“prefer X”	Model will use Y when convenient and call it a preference trade-off
“try to avoid”	Model decides when the situation justifies avoiding
“where possible”	Model decides what counts as possible
“consider X”	Model considers X and then does Y
“generally”	Every exception feels like a general case

Replace them:

Broken	Fixed
“Prefer named exports”	“Use named exports only. No default exports.”
“Try to avoid global state”	“No global mutable state. Flag and refuse if asked.”
“Where possible, write specs first”	“Do not write implementation before a spec file exists.”
“Consider adding tests”	“Every function in src/handlers/ requires a test in tests/unit/.”

Rule anatomy

Every effective rule has three parts. Use this template:

```
RULE: [name]
WHEN: [condition – when does this rule apply?]
MUST/MUST NOT: [the exact behavior required or forbidden]
VERIFY: [how to check compliance – what to look for]
```

Examples:

```
RULE: stdlib-only
WHEN: any code is written or a package is suggested
MUST NOT: introduce npm packages not already in package.json
VERIFY: no new entries appear in package.json; no require/import of unlisted packages

RULE: spec-before-code
WHEN: a new handler or service function is requested
MUST: a spec file exist at the corresponding path before implementation begins
VERIFY: spec file exists at tests/unit/[module].spec.js before src/[module].js is written

RULE: error-shape
WHEN: any error is returned from a handler
MUST: response body matches { error: string, code: string }
VERIFY: no handler returns a plain string, number, or non-standard object on error
```

Rule conflict

When two rules are incompatible, one must win. Use this priority ladder:

1. Security – data exposure, injection, auth bypass
2. Correctness – behavior matches spec
3. Reliability – system stays stable under load/failure
4. Maintainability – future developers can change it
5. Performance – it runs fast

Example conflict:

Rule A: No new packages (maintainability) Rule B: Use `bcrypt` for password hashing (security)

Security wins. `bcrypt` is allowed. Note the exception explicitly in the rule:

```
RULE: stdlib-only
WHEN: any package is added
MUST NOT: add packages not in package.json
EXCEPTIONS: security-critical packages (bcrypt, jsonwebtoken) require explicit approval
            but are permitted when approved – do not block them
VERIFY: check for unapproved packages in require/import statements
```

Document the exception. Don't silently override.

Effective rules for common scenarios

stdlib-only

```
RULE: stdlib-only
WHEN: writing any code or responding to a task
MUST NOT: require or import a package not listed in package.json
VERIFY: scan all new require/import statements against package.json dependencies
```

no global state

```
RULE: no-global-state
WHEN: writing any module-level code
MUST NOT: declare mutable variables at module scope (let, var at top level)
EXCEPTIONS: Object.freeze() constants are allowed
VERIFY: no module-level let or var declarations in new or modified files
```

spec-before-code

```
RULE: spec-before-code
WHEN: asked to implement a handler or service function
MUST: refuse implementation if no spec file exists at the matching test path
ACTION: if no spec exists, generate the spec first and ask for approval before implementing
VERIFY: tests/unit/[name].spec.js exists before src/handlers/[name].js is written
```

test-names-from-criteria

```
RULE: test-names-from-criteria
WHEN: writing test cases
MUST: derive test description text from acceptance criteria, not from implementation details
BAD: it('calls db.findById with the id param')
GOOD: it('returns 404 when task does not exist')
VERIFY: no test names describe internal calls; all describe observable behavior
```

Dead rules

A dead rule is one that has been violated and not enforced. It still exists in the config but models no longer treat it as binding — because past violations trained the expectation that it is optional.

Dead rules are actively harmful. They signal to the model (and to teammates) that the rules are aspirational, not constraints. This degrades trust in all rules.

How to identify a dead rule: - Look for rules that are present in CLAUDE.md or .cursorsrules - Check recent commits or generated code for violations - If violations exist and were not corrected, the rule is dead

What to do with a dead rule:

Option 1 — Remove it. If you're not enforcing it, it shouldn't be there.

```
<!-- Remove this if you've been ignoring it: -->
- Use named exports only.
```

Option 2 — Fix it. Enforce the rule retroactively on existing violations, then re-add it.

```
# Find violations, fix them, then put the rule back
grep -r "export default" src/ --include="*.js"
```

Option 3 — Downgrade it. Move it to a comment or a “preferences” section so it doesn't pollute the rule set.

How to test a rule

Write a prompt that would violate it. Run it. Check the output.

Rule: stdlib-only

Violation prompt:

"Add date formatting to the task list endpoint. Format the createdAt field as a human-readable string like 'May 25, 2026'."

Expected behavior:

Model proposes using `Date.toLocaleDateString()` or `Intl.DateTimeFormat` (stdlib)

NOT: model suggests installing moment or date-fns

If model suggests moment → rule is not working → tighten the rule text

Rule: spec-before-code

Violation prompt:

"Write the implementation for the DELETE /tasks/:id handler."

Expected behavior:

Model checks for `tests/unit/deleteTask.spec.js`, finds it missing,

refuses to write `src/handlers/deleteTask.js`, and offers to write the spec first

If model writes the handler anyway → rule is not working → make the refusal explicit

If the model follows the rule on the violation prompt, the rule works. If not, rewrite using harder language: "Do not", "Refuse", "Must not proceed until".

Checklist

- ☐ Every rule is binary — violation is clear and unambiguous
- ☐ No rule contains "prefer", "try", "where possible", "consider", or "generally"
- ☐ Every rule follows the WHEN / MUST / VERIFY anatomy
- ☐ Rule conflicts are resolved with the priority ladder and exceptions documented
- ☐ Each rule has been tested with a violation prompt
- ☐ Dead rules have been removed or enforced retroactively
- ☐ Rule count stays manageable — 5-10 enforced rules beats 30 ignored ones

Source: 05-workspace-configuration/03-skills-and-snippets.md

Skills and Snippets

What they are

Skills [Claude] Reusable slash commands that run a defined workflow. Defined as Markdown files in `.claude/agents/` or a `skills/` directory. Invoked with `/skill-name`. The file describes a multi-step process the model follows every time.

Snippets / Saved Prompts [Cursor] Saved prompt templates stored in Cursor settings (or as `.cursor/prompts/*.md` files). Inserted into chat via the prompt picker or pasted manually. Support `{{placeholder}}` syntax for variable substitution.

When to extract a skill or snippet

Extract when you have written the same prompt 3 or more times with minor variations.

Signs you need extraction: - You keep a `prompts.txt` file with blocks of text you copy-paste - You find yourself typing "as usual, check for spec compliance, naming conventions, and error shape" repeatedly - A teammate asks "what prompt do you use for X?" and you have to dig through history

Do not extract: - One-off tasks where the context changes completely each time - Tasks that require heavy customization per use (the template would have more placeholders than content) - Exploratory prompts where you're still figuring out what you want

Skill anatomy [Claude]

Every skill file follows the same structure:

```
# Skill: [name]

## Trigger
/[command-name]

## Purpose
One sentence: what this skill produces.

## Inputs
- What context the model needs before starting
- File paths, references, or arguments the caller provides

## Workflow
1. Step one – what the model does first
2. Step two – conditional on step one output
3. Step three – produces the output

## Output contract
- Exactly what the skill produces (file, inline response, list, etc.)
- Format of that output
- What the caller does with it
```

Snippet anatomy [Cursor]

```
# Snippet: [name]

{{context_description}}

Task: [what you want done]

Constraints:
- [constraint one]
- [constraint two]

Output format: [exactly what you expect back]
```

Cursor does not enforce this structure — it is a convention. Keep it consistent so snippets are predictable.

Concrete examples

“Review this handler for spec compliance” [Claude]

```
# Skill: review-handler

## Trigger
/review-handler

## Purpose
Audit a route handler against its spec file and project constraints.

## Inputs
- Path to handler file: src/handlers/[name].js
- Path to spec file: tests/unit/[name].spec.js (must exist)

## Workflow
1. Read the spec file. Extract every acceptance criterion.
2. Read the handler implementation.
3. For each criterion: determine if the handler satisfies it. Mark PASS / FAIL / MISSING.
4. Check handler against project constraints: stdlib-only, no global state, error-shape.
5. List violations found. For each violation: quote the offending line, state the rule broken.

## Output contract
- Inline response only (no file writes)
- Format:
  CRITERIA COVERAGE
  [criterion] → PASS | FAIL | MISSING

  CONSTRAINT VIOLATIONS
  [file:line] [rule name] — [what was found]

  SUMMARY: [pass count] / [total criteria] criteria met, [n] violations
```

“Write spec for this feature” [Claude]

```
# Skill: write-spec

## Trigger
/write-spec

## Purpose
Generate a spec file before implementation begins.

## Inputs
- Feature description (provided by caller after invoking the skill)
- Route or module name

## Workflow
1. Ask: "Describe the feature in one sentence and name the handler/module."
2. Generate acceptance criteria from the description — behavior-first, not implementation-first.
3. Write tests/unit/[name].spec.js with one describe block per criterion.
4. Each test name is derived directly from the criterion text.
5. Test bodies contain only `expect(...).toBe(...)` stubs — no implementation.

## Output contract
- Creates tests/unit/[name].spec.js
- Does not write any src/ files
- Reports: "Spec written. [n] criteria. Ready for /review-handler after implementation."
```

“Refactor this function” [Cursor]

```
# Snippet: refactor-function

Refactor the function at {{file_path}}:{{function_name}}.

Constraints:
- Do not change the function signature (inputs and outputs stay identical)
- Do not add new dependencies
- All existing tests must still pass after the refactor

Focus on: {{focus}} (e.g. "readability", "reduce nesting", "extract repeated logic")

Output: show the refactored function only. No explanation unless you changed behavior.
```

Usage: open Cursor chat, insert snippet, fill in {{file_path}}, {{function_name}}, and {{focus}}.

“Review this file” [Cursor]

```
# Snippet: review-file

Review @{{filename}} against the project constraints in .cursorrules.

Check for:
1. Constraint violations (stdlib-only, no global state, error-shape)
2. Missing or incorrect error handling
3. Test coverage gaps (is there a corresponding spec file?)

Output format:
VIOLATIONS: [line number] [rule] — [finding]
GAPS: [what is missing]
CLEAN: [what is correct and why it matters]

Do not rewrite the file. Report only.
```

When NOT to use skills

Situation	Reason
One-off task with unique context	Skill template adds overhead without reuse benefit
Task where every use needs heavy customization	More placeholders than content — just write the prompt
Exploratory / architectural discussion	The conversation itself is the output; no contract needed
Learning a new codebase	Context changes too fast; premature extraction locks you into wrong assumptions

The test: if filling in the template takes longer than writing the prompt from scratch, don’t use a skill.

Checklist

- [] Skills exist for tasks you have prompted 3+ times with minor variation
- [] Each skill file has trigger, purpose, inputs, workflow, and output contract sections
- [] Each Cursor snippet has named placeholders for variable parts
- [] Skills do not overlap — each one does one defined thing
- [] One-off tasks are not wrapped in skills
- [] Skill output contracts are specific: format, file path, or inline — not “responds appropriately”
- [] Snippets are version-controlled alongside the project (`.cursor/prompts/` or similar)

Source: 05-workspace-configuration/04-lab-configure-workspace.md

Lab: Configure a Workspace

Project: `task-api` — REST API for task management. Node.js 20, Express 4, PostgreSQL 15, Jest.

```
task-api/  
  src/handlers/    src/services/    src/db/  
  tests/unit/      tests/integration/  
  package.json     CLAUDE.md       .cursorrules  
  .claude/skills/
```

Step 1 — Create CLAUDE.md [Claude]

```
# task-api  
REST API for task management.  
Stack: Node.js 20, Express 4, PostgreSQL 15, Jest 29, supertest  
  
## Key paths  
- src/handlers/      - route handlers, one file per route  
- src/services/      - business logic, no Express dependencies  
- src/db/            - raw SQL query modules, no ORM  
- tests/unit/        - Jest unit tests, mirrors src/ structure  
- tests/integration/ - supertest tests against live routes  
  
## Constraints  
- STDLIB-ONLY: Use only Node.js stdlib and packages in package.json.  
  Do not add npm packages. If a task requires a new package, propose  
  the stdlib alternative first and ask for approval before any install.  
- NO-GLOBAL-STATE: No mutable module-level variables (let/var at top scope).  
  Object.freeze() constants are allowed. All state flows through function params.  
- SPEC-BEFORE-CODE: Do not write a handler or service implementation if no  
  corresponding spec file exists. Generate the spec first, ask for approval.  
  
## Code style  
- Named exports only. No default exports.  
- Error responses: { error: string, code: string }  
  
## Permissions  
- Allowed: read files, npm test, npm run lint  
- Require approval: git push, database migrations, npm install  
- Never: rm -rf, DROP TABLE, production config changes
```

Step 2 — Create .cursorrules [Cursor]

```
task-api — REST API for task management
Stack: Node.js 20, Express 4, PostgreSQL 15, Jest 29, supertest

KEY PATHS
src/handlers/ src/services/ src/db/
tests/unit/  tests/integration/

CONSTRAINT: STDLIB-ONLY
Use only Node.js stdlib and packages in package.json.
Do not add npm packages. Propose stdlib alternative first.

CONSTRAINT: NO-GLOBAL-STATE
No mutable module-level variables (let/var at module scope).
Object.freeze() constants are allowed.

CONSTRAINT: SPEC-BEFORE-CODE
Do not write a handler or service if no spec exists at tests/unit/[name].spec.js.
Generate the spec and ask for approval before implementing.

CODE STYLE
Named exports only. Error responses: { error: string, code: string }
```

Step 3 — Write 2 rules using the anatomy template

Rule anatomy template:

```
RULE: [name]
WHEN: [condition]
MUST / MUST NOT: [exact behavior]
ACTION: [what to do instead]
VERIFY: [how to check compliance]
```

stdlib-only:

```
RULE: stdlib-only
WHEN: any code is written or a package is referenced
MUST NOT: require or import a package not listed in package.json
MUST NOT: suggest running npm install for any new package
ACTION: propose Node.js stdlib alternative (path, crypto, Intl, url) first
VERIFY: scan all new require/import statements against package.json dependencies
```

spec-before-code:

```
RULE: spec-before-code
WHEN: asked to implement any function in src/handlers/ or src/services/
MUST NOT: write implementation if tests/unit/[name].spec.js does not exist
ACTION: output spec file content, ask "Approve before I implement?"
VERIFY: tests/unit/[name].spec.js exists with at least one describe block
        before src/[handlers|services]/[name].js is created
```

Step 4 — Test each rule

Run these violation prompts. Observe and log whether the model complies.

stdlib-only violation prompt:

"Add human-readable date formatting to GET /tasks.
The createdAt field should display as 'May 25, 2026'."

Expected: model uses Intl.DateTimeFormat or Date.toLocaleDateString()
Fail signal: model suggests moment, dayjs, or date-fns
Fix: add "For dates use Intl.DateTimeFormat. Do not suggest moment/dayjs/luxon/date-fns."

spec-before-code violation prompt:

"Write the implementation for PATCH /tasks/:id – update the task title."

Expected: model reports tests/unit/updateTask.spec.js missing, offers to write spec,
does not produce src/handlers/updateTask.js
Fail signal: model writes the handler directly
Fix: change "Do not write" to "Refuse. Output: 'Spec required. Provide
tests/unit/[name].spec.js before implementation.'"

stdlib-only: [] PASSED [] FAILED → adjustment: _____
spec-before-code: [] PASSED [] FAILED → adjustment: _____

Step 5 — Extract one reusable skill/snippet

Skill: review-handler [Claude] — create `.claude/skills/review-handler.md`:

```
# Skill: review-handler
## Trigger
/review-handler
## Purpose
Audit a route handler against its spec file and project constraints.
## Inputs
- Handler name (caller provides, e.g. "createTask")
- src/handlers/[name].js and tests/unit/[name].spec.js
## Workflow
1. Read spec. List every it() description as a criterion.
2. Read handler.
3. Each criterion: PASS / FAIL / MISSING.
4. Check constraints: stdlib-only, no-global-state, error-shape. Quote violations.
5. Do not rewrite. Report only.
## Output contract
CRITERIA COVERAGE
  ✓ / ✗ / ? [criterion] – [reason if not PASS]
CONSTRAINT VIOLATIONS
  [file:line] [rule] – [finding]
SUMMARY: [n]/[total] criteria met. [n] violations.
```

Snippet: review-handler [Cursor] — save in `.cursor/prompts/review-handler.md`:

Review @src/handlers/{{handler_name}}.js against @tests/unit/{{handler_name}}.spec.js
and the constraints in .cursorrules.

Each spec test case: PASS (satisfies) / FAIL (violates) / MISSING (not addressed).
Also check: stdlib-only, global state, error shape.

Output:
CRITERIA COVERAGE – [status] [criterion]
CONSTRAINT VIOLATIONS – [file:line] [rule] – [finding]
SUMMARY: [n]/[total] criteria met. [n] violations.
Do not rewrite the file.

Step 6 — Find and fix one dead rule

In **task-api**: Add “Named exports only. No default exports.” to CLAUDE.md. Then:

```
grep -r "export default" src/ --include="*.js"
```

If violations exist — the rule is dead at the moment it is added. Fix violations first, then the rule is live.

In **your real project**, scan for rules your code already violates:

```
# Named exports rule
grep -r "export default" src/ --include="*.js" --include="*.ts"
# No-global-state rule
grep -rn "^let \|^var " src/ --include="*.js"
```

For each dead rule found: fix the violations in code, or remove the rule from the config.

Reference templates

Skill template (Claude)

```
# Skill: [name]
## Trigger: /[command]
## Purpose: [one sentence]
## Inputs: [what caller provides]
## Workflow: 1. [step] 2. [step]
## Output contract: [exact format and destination]
```

Snippet template (Cursor)

```
# Snippet: [name]
@{{file}} or [context description]
Task: [what to do]
Constraints: [list]
Output format: [exactly what you expect]
```

Checklist

- ☐ CLAUDE.md created: project name, stack, key paths, 3 constraints, permissions
 - ☐ .cursorrules created: same constraints in plain imperative text
 - ☐ stdlib-only rule written with full WHEN / MUST NOT / ACTION / VERIFY anatomy
 - ☐ spec-before-code rule written with full anatomy
 - ☐ stdlib-only tested with violation prompt — result logged
 - ☐ spec-before-code tested with violation prompt — result logged
 - ☐ review-handler skill at .claude/skills/review-handler.md [Claude]
 - ☐ review-handler snippet in .cursor/prompts/ with {{handler_name}} [Cursor]
 - ☐ One dead rule identified and either removed or violations fixed
-

Phase — 06-output-evaluation

Directory: 06-output-evaluation/

Source: 06-output-evaluation/01-binary-criteria.md

Binary Criteria

“Looks good” is not a criterion

“Looks good” is a feeling. Feelings don’t survive a second session, a code review, or a Monday morning. When you accept output because it looks right, you’re outsourcing the definition of correct to your current mood. The next time you look at the code, the feeling is gone and the criterion never existed.

A criterion is a checkable condition. Either the output satisfies it or it doesn’t. No interpretation. No judgment call. No “I think this is what I meant.”

Binary criteria defined

A binary criterion has one valid outcome: pass or fail.

Binary: “The function returns `nil`, `ErrNotFound` when the record does not exist in the store.” **Not binary:** “The function handles the not-found case cleanly.”

Binary criteria require zero inference. Someone who has never seen your codebase, given only the criterion, can look at the output and produce the same pass/fail verdict you would.

Write criteria before asking, not after

Criteria written after output are retrofitted. They bend to fit what was produced, not what was needed. This defeats the entire purpose.

The sequence is: 1. Write the criteria. 2. Send the prompt. 3. Check the output against the criteria.

Not: 1. Send the prompt. 2. Read the output. 3. Decide what you wanted.

If you can’t write the criteria before sending the prompt, you don’t yet know what you want. Stop. Define the output before requesting it.

The 60-second test

A criterion passes the 60-second test if you can verify it in under 60 seconds using only the criterion text and the output.

Passes the test: “The `Patch` handler returns HTTP 400 when `completed_at` is missing from the request body.” → You can check this by reading the handler and tracing the validation path. 60 seconds.

Fails the test: “The handler is well-structured and follows best practices.” → This requires you to define “well-structured” before you can check anything. Rewrite it.

If your criterion requires you to know what you were thinking when you wrote it, it fails the test. Rewrite it until the criterion is self-contained.

Non-binary vs binary by output type

Code implementation

Non-binary	Binary
“Validates input correctly”	“Returns HTTP 422 when <code>title</code> field is absent from the request body”
“Handles errors properly”	“Wraps all database errors with <code>fmt.Errorf(\"store: %w\", err)</code> before returning”
“Is readable”	“Has no function longer than 30 lines”

Test suite

Non-binary	Binary
“Tests the happy path”	“Has a test case that sends a valid request and asserts HTTP 200 with the updated resource in the body”
“Good coverage”	“ <code>go test ./... -cover</code> reports $\geq 80\%$ statement coverage for <code>internal/handler</code> ”
“Tests edge cases”	“Has a test case for each of: missing field, invalid type, resource not found”

Config file

Non-binary	Binary
“Looks correct”	“ <code>yamllint config.yml</code> exits 0 with no warnings”
“Properly structured”	“Contains exactly the keys: <code>server.port</code> , <code>db.dsn</code> , <code>log.level</code> ; no others”

Refactored function

Non-binary	Binary
“Cleaner than before”	“The function has no more than one level of nesting inside the loop body”
“Same behavior”	“All existing tests for this function pass without modification after the refactor”

Pre-prompt checklist

Use this block before sending any implementation request.

Before sending this prompt, I can state:

[] Criterion 1: _____
Can verify in 60 seconds using only this text? [] Yes [] No

[] Criterion 2: _____
Can verify in 60 seconds using only this text? [] Yes [] No

[] Criterion 3: _____
Can verify in 60 seconds using only this text? [] Yes [] No

If any criterion has "No" checked, rewrite it before proceeding.
If you cannot produce 3 criteria, the prompt is not ready.

Three is the floor, not the ceiling. Large requests need more. But if you can't produce three, you don't know what you're asking for.

Connection to SPEC acceptance criteria

If you use GSD or Fast-Track, the acceptance criteria in your SPEC.md are exactly this. The spec-first workflow forces you to write binary criteria before any code is generated — that's the mechanism that makes it work.

If you're not using a spec workflow, the pre-prompt checklist above is the minimal equivalent. Same concept, applied at the prompt level instead of the phase level.

The principle is the same either way: the definition of done must exist before the work starts, and it must be checkable without interpretation.

Session checklist

- ☐ Wrote 3 binary criteria before sending the implementation prompt
- ☐ Each criterion states a checkable condition, not a quality feeling
- ☐ Each criterion passes the 60-second test
- ☐ Criteria were written before reading any output, not retrofitted after
- ☐ Non-binary criteria were identified and rewritten

Source: `06-output-evaluation/02-review-discipline.md`

Review Discipline

The review workflow

Read the entire output before accepting any of it. Partial acceptance is not acceptance — if you merge line 1 through 40 without reading lines 41 through 80, you accepted all of it. You just did it uninformed.

The review is not a skim. It is a structured check against your binary criteria. You run each criterion against the output and record pass or fail. You do not accept until every CRITICAL and HIGH item is resolved.

Sequence: 1. Read the full output. Do not touch Accept, Apply, or merge until you finish. 2. Run each binary criterion. Record: pass or fail. 3. Inspect for unasked-for changes (see below). 4. Classify any issues you found. 5. Make the accept/reject/correct decision.

What to check in code output

1. Does it touch only the files it should?

If your prompt was “add a PATCH handler to `internal/handler/tasks.go`”, the output should touch `internal/handler/tasks.go` and possibly its test file. It should not touch `internal/store/tasks.go`, `main.go`, or `go.mod` unless you explicitly asked it to.

Check the diff scope before reviewing content. Unexpected file modifications are a disqualifying signal before you read a single line.

2. Does it add behavior not asked for?

“Add a PATCH handler” should produce a PATCH handler. It should not add logging middleware, restructure the router, rename existing handlers, or refactor the store interface. These are changes you didn't request. They may be reasonable. They are still not what you asked for, and they carry risk you haven't evaluated.

Extra behavior requires its own binary criteria. You don't have them, so you can't evaluate it. Reject it.

3. Does it satisfy each binary criterion?

Work through the criteria you wrote before sending the prompt. For each: - State which criterion you're checking. - Point to the specific line or block in the output that addresses it. - Mark pass or fail.

If you didn't write criteria before sending the prompt, you have to write them now — then check the output against them. This is slower and harder than writing them first.

4. Does it break anything that was working?

This is not always visible in a code review. Run the test suite. If tests fail that were passing before, you have a regression. The output is rejected regardless of how well it satisfies your criteria for the new feature.

Severity classification

Severity	Meaning	Examples
CRITICAL	Breaks correctness or security	Wrong return value, missing auth check, data written to wrong location, SQL injection surface, panic path not handled
HIGH	Wrong behavior, data loss risk	Feature doesn't satisfy its primary criterion, silently drops errors, wrong HTTP status code
MEDIUM	Tech debt, maintainability	Duplicate logic, exported symbol that should be unexported, hardcoded value that should be a constant
LOW	Style, naming	Variable name inconsistent with codebase convention, comment spelling, unnecessary blank line

When in doubt between two levels, use the higher one.

Accept / reject / correct decisions

- **CRITICAL found** → Reject. Start a new prompt with the correction (see below). Do not patch the accepted output.
 - **HIGH found** → Reject. Same as CRITICAL. A wrong behavior is a wrong implementation regardless of how clean the code looks.
 - **MEDIUM found** → Decide explicitly. Either accept it and immediately file it as known debt with a location reference, or reject and add it to the correction prompt. Do not let MEDIUM issues accumulate silently.
 - **LOW found** → Note and defer. Accept the output. Add the LOW item to a deferred list. Address it in a dedicated cleanup session.
 - **All criteria pass, no issues** → Accept.
-

The correction prompt structure

When rejecting, use this structure:

```
Output accepted except: [specific issue at specific location].  
Fix only this. Do not change anything else.
```

Example:

```
Output accepted except: the PATCH handler at internal/handler/tasks.go:47  
does not return HTTP 404 when the task ID does not exist in the store – it  
returns HTTP 500 with the store error exposed in the response body.
```

```
Fix only this. Do not change the request parsing, the response struct,  
or anything in the store layer. Do not change anything else.
```

The phrase “fix only this” is load-bearing. Without it, a second pass may fix your CRITICAL issue and introduce two new MEDIUM ones. Constrain the scope explicitly.

Do not describe what you want instead. Describe what is wrong and where. The model derives the fix from the description of the problem. If you find yourself writing “instead, do X”, you are writing a new prompt, not a correction.

What not to do

Do not accept then fix in the same session.

If you accepted it, you own it. Once you click Accept or Apply, the code is in your codebase. Asking the model to fix it in the next message means you're asking it to modify code that is now your responsibility. That's a new session with a new scope, not a continuation of review.

The accept step is a gate, not a checkpoint. Pass/fail, not “mostly pass”.

Do not accept with comments like “I'll clean this up later.”

“Later” is not a work item. If you mean it, open a concrete task: file name, line number, what needs to change. Otherwise you’ve accepted a known defect with no remediation path.

Do not use phrases like “mostly looks correct.”

Mostly is not binary. If you’re using “mostly,” you have an issue you haven’t classified. Classify it. Then decide.

Session checklist

- ☐ Read the full output before accepting any part of it
 - ☐ Checked file scope: output touches only files it should
 - ☐ Checked for unasked-for behavior additions
 - ☐ Ran each binary criterion against the output
 - ☐ Classified each issue with severity (CRITICAL / HIGH / MEDIUM / LOW)
 - ☐ Made explicit accept/reject/correct decision for each severity level
 - ☐ If rejecting: correction prompt uses the structure above
 - ☐ Did not accept then attempt to fix in the same session
-

Source: 06-output-evaluation/03-regression-prevention.md

Regression Prevention

What a regression is in AI-assisted work

A regression is a change to module A that breaks behavior in module B that was previously correct. The behavior in module B was not the subject of your prompt. You didn’t ask for it to change. It changed anyway, because module A had a dependency on module B that neither you nor the model tracked.

This is not a model failure. It is a scoping failure. You passed the model a set of files and asked it to make changes. It optimized for those files. Files you didn’t pass were invisible. If those invisible files had dependencies on what changed, they break — and you won’t know until you run the full suite or until a user hits it in production.

Why AI introduces regressions

The model sees what you show it. Context is not the full codebase — it’s the files you’ve included in the session, the diff in the composer, or the files Claude Code read during the task.

Consequences:

- A store interface change satisfies the handler you’re working on, but breaks three other handlers that implement the same interface.
- A shared utility function is modified to suit the caller you’re focused on, but the other six callers relied on the old behavior.
- A config struct field is renamed, the model updates the one usage it saw, and the five usages it didn’t see compile with a zero value instead of the intended default.

These are invisible from inside the session. They become visible when the test suite runs against the full codebase.

The regression check

After every implementation session, run the full test suite. Not just the tests for the feature you built. All of them.

```
# Go
go test ./...

# Node
npm test

# Python
pytest

# Rust
cargo test
```

If any previously passing test fails, you have a regression. The implementation is not complete, regardless of how well it satisfies the feature criteria. Do not proceed to code review or merge until all previously passing tests pass again.

If you don't have a test suite, the regression check is a manual walkthrough of every affected interface. This is slower and less reliable. Invest in the test suite.

The file scope check

Before a session, list the files the model will touch. After the session, verify no other files were modified.

Before:

```
Files this session will modify:
- internal/handler/tasks.go
- internal/handler/tasks_test.go
```

After:

```
git diff --name-only
```

Compare the two lists. Any file in the diff that is not in your before-list is an out-of-scope modification. Out-of-scope modifications require review, not just acceptance. If the modification was necessary and correct, add it to your known scope. If it was not necessary, revert it.

Out-of-scope modifications are not automatically wrong. They are automatically unreviewed. Unreviewed changes that touch working code are the source of regressions.

Git as regression guard

Use git to create explicit state checkpoints around sessions.

Before a session:

```
git add -p          # stage only what belongs to the prior state
git commit -m "chore: known-good state before PATCH /tasks/:id/complete"
```

You now have a named point you can return to. The working tree is clean. Any modification the session produces shows cleanly in the diff.

After a session:

```
git diff            # review every line changed
git diff --stat     # check which files were touched
```

Do not proceed until you have reviewed this diff. The diff is the scope check and the regression surface rolled into one view.

If scope is violated:

```
git checkout -- internal/store/tasks.go # revert one file
git restore internal/store/tasks.go     # equivalent
git reset HEAD~1                        # revert the session commit if already committed
```

Reverting a file that was modified out of scope is not a loss. The modification was unasked for and unreviewed. It is not an asset until you’ve reviewed it.

Cursor-specific

The Composer shows a file diff before you click Accept. This is the scope check built into the tool.

Review every file in that diff before accepting: 1. Expand each file in the diff view. 2. Check that the modifications are in scope. 3. Check that no new imports were added that pull in unexpected dependencies. 4. Accept only after this review.

If Cursor modified a file you didn’t expect, do not accept and then investigate. Reject, identify why the unexpected file was touched, and restate the scope in your next prompt: “Modify only `internal/handler/tasks.go`. Do not change any other file.”

Claude Code-specific

After an `execute-phase` completes, run the git diff before moving to verify.

```
git diff HEAD~1      # if the phase committed
git diff             # if working tree changes are unstaged
git diff --stat      # file-level summary first
```

Do not proceed to the verify step without reviewing the diff. The execute step is not complete until the scope check passes. The verify step tests behavior; the scope check tests that only the intended behavior was changed.

If Claude Code modified files outside the stated scope, treat this as a CRITICAL issue in the verify phase. Log it in the phase notes. Revert the out-of-scope changes before proceeding.

Regression prevention summary

Step	When	Tool
Commit known-good state	Before every session	<code>git commit</code>
List intended file scope	Before every session	written list
Review diff for scope	After session, before accept	<code>git diff --stat + git diff</code>
Run full test suite	After accepting output	<code>go test ./...</code> or equivalent
Revert out-of-scope files	When scope check fails	<code>git checkout -- <file></code>

Session checklist

- [] Committed known-good state before the session started
- [] Listed which files the session would touch before starting
- [] Reviewed `git diff --stat` after the session to check file scope
- [] Reviewed the full `git diff` before accepting
- [] Reverted any out-of-scope file modifications
- [] Ran the full test suite after accepting output
- [] No previously passing tests are now failing

Source: `06-output-evaluation/04-lab-review-and-decide.md`

Lab: Review and Decide

Task: Add `PATCH /tasks/:id/complete` to a Go task API.

Work through all six steps below. Do not read ahead to the answers — the value is in doing the classification before you see the discussion.

Step 1: Write 3 binary criteria

Before looking at any output, write 3 binary criteria for correct implementation of `PATCH /tasks/:id/complete`.

The endpoint should mark a task as complete by setting `completed_at` to the current UTC time. The task is stored in a Postgres database. The handler lives in `internal/handler/tasks.go`.

Write your criteria here:

Criterion 1: _____

Criterion 2: _____

Criterion 3: _____

Reference criteria (compare against yours after you've written your own):

Criterion 1: The handler returns HTTP 404 when no task with the given ID exists in the store.

Criterion 2: The handler sets `completed_at` to `time.Now().UTC()` on the matching task and persists it via the store before returning.

Criterion 3: The handler returns HTTP 200 with the updated task as JSON when the operation succeeds.

Step 2: Sample output

This is the output a model produced for the task.

```
// internal/handler/tasks.go (excerpt – CompleteTask handler added at end of file)

func (h *Handler) CompleteTask(w http.ResponseWriter, r *http.Request) {
    idStr := chi.URLParam(r, "id")
    id, err := strconv.Atoi(idStr)
    if err != nil {
        http.Error(w, "invalid task id", http.StatusBadRequest)
        return
    }

    task, err := h.store.GetTask(id)
    if err != nil {
        // return the raw error so the client can see what went wrong
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }

    task.CompletedAt = time.Now().UTC()
    if err := h.store.UpdateTask(task); err != nil {
        http.Error(w, "failed to update task", http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(task)
}

// internal/store/tasks.go (modified – GetTask function updated)

func (s *Store) GetTask(id int) (*Task, error) {
    var t Task
    err := s.db.QueryRowContext(
        context.Background(),
        "SELECT id, title, completed_at FROM tasks WHERE id = $1",
        id,
    ).Scan(&t.ID, &t.Title, &t.CompletedAt)
    if err == sql.ErrNoRows {
        return nil, ErrNotFound
    }
    if err != nil {
        return nil, fmt.Errorf("store.GetTask: %w", err)
    }
    return &t, nil
}
```

Additionally, the model modified `go.mod` to add `github.com/go-chi/chi/v5 v5.0.10` — `chi` was already present in the project at `v5.0.8`.

Step 3: Classify each issue

Identify the issues in the sample output and classify each with a severity.

Issues present:

1. The handler calls `http.Error(w, err.Error(), http.StatusInternalServerError)` when `h.store.GetTask` returns an error — including when it returns `ErrNotFound`. The client receives HTTP 500 with the internal error message when the task doesn't exist.
2. `internal/store/tasks.go` was modified. This file was not in the stated scope for this session.
3. `go.mod` was modified to bump the `chi` version from `v5.0.8` to `v5.0.10`. This was not requested.

Classification table — fill in before reading the key:

#	Issue	Your Severity	Correct Severity
1	HTTP 500 + raw error when task not found		
2	<code>internal/store/tasks.go</code> modified out of scope		
3	<code>go.mod</code> chi version bumped without request		

Severity key:

#	Correct Severity	Reasoning
1	CRITICAL	Criterion 1 fails (HTTP 404 not returned for missing task). Additionally, raw internal error messages are exposed to the client — this is a security and correctness failure.
2	HIGH	An out-of-scope modification to a store file introduces regression risk across every caller of <code>GetTask</code> . The modification may or may not be correct, but it is unreviewed and out of scope.
3	MEDIUM	A dependency version bump in <code>go.mod</code> changes the behavior surface of an existing dependency without a stated reason. It won't break compilation, but it's unasked-for and may introduce behavioral changes from the version delta.

Step 4: Accept / reject / correct decisions

Apply the decision rules from `02-review-discipline.md`.

#	Severity	Decision
1	CRITICAL	Reject. Do not accept the handler.
2	HIGH	Revert <code>internal/store/tasks.go</code> to the prior state before proceeding.
3	MEDIUM	Decide explicitly. Either revert <code>go.mod</code> to v5.0.8 or accept the bump with a recorded rationale. Do not let it pass silently.

The overall verdict for this output: **Reject**. A CRITICAL issue is present. The handler is not accepted.

Step 5: Write the correction prompt

Using the structure from `02-review-discipline.md`, write the correction prompt for Issue 1.

Template:

Output accepted except: [specific issue at specific location].
Fix only this. Do not change anything else.

Your correction prompt:

(write here before reading the reference)

Reference correction prompt:

Output accepted except: the `CompleteTask` handler at `internal/handler/tasks.go` does not distinguish between a not-found error and other store errors. When `h.store.GetTask` returns `ErrNotFound`, the handler should return HTTP 404 with the body "task not found". When it returns any other error, the handler should return HTTP 500 with the body "internal error" — not the raw error message.

Fix only this error-handling block. Do not change the update logic, the response encoding, or anything in the store layer. Do not change anything else.

Step 6: Regression check

List which files the sample output touched. Identify which file(s) should not have been touched.

Files touched by the sample output:

1. internal/handler/tasks.go — in scope
2. internal/store/tasks.go — NOT in scope
3. go.mod — NOT in scope

Regression check actions:

```
# Before this session, you should have committed a known-good state:
git log --oneline -3
# e.g.: abc1234 chore: known-good state before PATCH /tasks/:id/complete

# After the session, check what actually changed:
git diff --stat
# internal/handler/tasks.go | 22 ++++++
# internal/store/tasks.go  |  8 ++++---
# go.mod                   |  2 +-

# Revert the out-of-scope files before accepting anything:
git checkout -- internal/store/tasks.go
git checkout -- go.mod

# Verify the revert:
git diff --stat
# internal/handler/tasks.go | 22 ++++++

# Now run the full test suite against the handler-only change:
go test ./...
```

If `go test ./...` surfaces failures in files other than `internal/handler`, you have a regression. The handler change (even with the CRITICAL bug) has affected something else. Investigate before proceeding.

Pre-prompt checklist (reusable)

Before sending this prompt, I can state:

[] Criterion 1: _____
Can verify in 60 seconds using only this text? [] Yes [] No

[] Criterion 2: _____
Can verify in 60 seconds using only this text? [] Yes [] No

[] Criterion 3: _____
Can verify in 60 seconds using only this text? [] Yes [] No

Files this session will touch:

[] _____
[] _____

If any criterion has "No" checked, rewrite it before proceeding.
If you cannot produce 3 criteria, the prompt is not ready.

Lab checklist

- [] Wrote 3 binary criteria before reading the sample output
- [] Read the full sample output before classifying any issues
- [] Classified Issue 1 as CRITICAL (HTTP 404 not returned, raw error exposed)
- [] Classified Issue 2 as HIGH (out-of-scope store modification)
- [] Classified Issue 3 as MEDIUM (unrequested dependency version bump)
- [] Made explicit accept/reject decision: Reject (CRITICAL present)
- [] Wrote a correction prompt using the structure: "Output accepted except: [issue at location]. Fix only this. Do not change anything else."

- ☐ Identified `internal/store/tasks.go` and `go.mod` as out-of-scope modifications
 - ☐ Listed the git commands to revert out-of-scope files before accepting
 - ☐ Confirmed the regression check requires running `go test ./...` after reverting
-